

Apollo AgriVerse

AgrIntel Division

Final End-Term Internship Technical Report

An Intelligent, Stateful and Explainable Agricultural Intelligence System

Project / Division	Apollo AgriVerse — AgrIntel Division
Organization	Chatake Innworks Pvt. Ltd.
Internship Period	June 2026 – August 2026
Supervisor	Akash Shivdas Chatake
Report Date	August 2026

From agricultural evidence to evolving state, predictive intelligence and actionable insight.

Prepared as the final technical record of the internship engineering work, including research, data engineering, digital-twin implementation, machine learning, backend services, persistence, frontend development, validation and deployment readiness.

CERTIFICATE

This is to certify that the project titled **Apollo AgriVerse — AgriIntel Division** was carried out during the internship period stated in this report under the guidance of **Akash Shivdas Chatake** at **Chatake Innworks Pvt. Ltd.**. The work documented herein represents the engineering activities, implementations, experiments and validation evidence completed by the project team up to the end-term submission.

The report is intended as a technical record. Implementation claims are tied to available artifacts and validation results, while future or production-level capabilities are identified separately.

Supervisor Signature: _____

Name: _____

Date: _____

DECLARATION

We declare that this report is an original technical account of the internship work performed by the project team. Sources, public datasets, software libraries and external technical material have been acknowledged where used. The report distinguishes implemented software, evaluated models, simulated behaviour, prototype interfaces and proposed future work so that the maturity of the system is not overstated.

Vijayalaxmi K. Sundalam — Technical Integration _____
Nandini N. Naral — Product, Data Quality & Mining _____
Sunaina S. Gaikwad — ML Baseline & Model Evaluation _____
Dakshini A. Neel — Data & ML-Model Frontend _____

ACKNOWLEDGEMENT

We express our sincere gratitude to **Akash Shivdas Chatake** for continuous technical guidance, review and direction throughout the internship. The project evolved substantially through iterative feedback, particularly the shift from isolated prediction and dashboard features toward a coherent agricultural system that accounts for soil, crop variety, environmental conditions, interventions and lifecycle state.

We also acknowledge Chatake Innoworks Pvt. Ltd. for providing the engineering environment in which the AgriIntel work package was developed and evaluated. We thank the creators of the public datasets and software libraries used during the project, and recognize the collaborative contribution of every team member across data, machine learning, backend integration, frontend development and technical documentation.

ABSTRACT

Apollo AgriVerse — AgriIntel is an intelligent agricultural system developed around the idea that farm intelligence should represent an evolving agricultural condition rather than operate as a collection of disconnected predictions. The final system package combines heterogeneous agricultural data, synthetic telemetry, explicit state-based digital-twin simulation, machine-learning pipelines, a persistence layer, backend service boundaries and a farmer-facing frontend dashboard. The engineering work progressed from problem research and data preparation to state modelling, digital-twin synchronization, predictive intelligence, API integration, persistence and interface development.

A central architectural decision was to separate temporal state representation from learned prediction. State models were used for variables whose condition persists and evolves—such as soil moisture, crop lifecycle stage, hydrogel capacity and mulch degradation—while machine learning was introduced where learned relationships are useful. The documented digital-twin engine uses differential state updates and stage-aware thresholds, with SQLite persistence and a recovery mechanism that allows the latest state to be reloaded after interruption. Five documented validation tests cover state continuity, causal intervention, scenario branching, database recovery and phenology-stage intelligence.

Two ML pipelines are documented as notebook-level evaluation evidence. A Random Forest grape-yield baseline achieved a held-out R^2 of -0.0942, indicating that it is not suitable for a production claim in its current form. A Gradient Boosting telemetry model achieved RMSE 0.9484%, MAE 0.1456% and R^2 0.9900 on its test split; this is a strong computational result but remains subject to broader validation. The final report therefore treats model quality, twin correctness, backend integration and field validation as separate engineering questions rather than collapsing them into a single “AI accuracy” claim.

Keywords: precision agriculture; digital twin; state model; machine learning; grape yield; telemetry; hydrogel; intelligent soil; smart mulching; FastAPI; SQLite; Growing Degree Days; agricultural decision support.

ABBREVIATIONS AND NOTATION

Term	Meaning
AgriIntel	Agricultural intelligence division / work package of Apollo AgriVerse
API	Application Programming Interface
ET	Evapotranspiration
GDD	Growing Degree Days
ML	Machine Learning
RF	Random Forest
GB	Gradient Boosting
SQLite	Embedded relational database used for prototype state persistence
DT	Digital Twin
UI	User Interface
REST	Representational State Transfer
NPK	Nitrogen, Phosphorus and Potassium
R ²	Coefficient of determination
RMSE	Root Mean Squared Error
MAE	Mean Absolute Error

Table 0.1 — Principal abbreviations used throughout the report.

TABLE OF CONTENTS

CERTIFICATE 2

DECLARATION 3

ACKNOWLEDGEMENT 4

ABSTRACT 5

ABBREVIATIONS AND NOTATION 6

TABLE OF CONTENTS 7

1. Introduction and Project Context 11

1.1 Project Background 11

1.2 Problem Definition 11

1.3 Final Engineering Objective 11

2. Requirements, Scope and Project Evolution 12

2.1 Requirements Maturation 12

2.2 Functional Requirements 12

2.3 Non-Functional Requirements 12

3. System Scope and Solution Strategy 13

3.1 AgriIntel Work Package 13

3.2 Solution Strategy 13

3.3 Evidence Classes 13

4. Data Acquisition and Engineering 14

4.1 Data Strategy 14

4.2 Dataset Inventory 14

4.3 Data Cleaning and Harmonization 14

4.4 Synthetic Data Generation 15

5. Digital Twin Architecture 16

5.1 Digital Twin Definition for AgriIntel 16

5.2 Twin State Vector 16

5.3 State Transition Principle 16

6. State Models and Agricultural Simulation 17

6.1 Why State Models Were Required 17

6.2 Alternatives Considered 17

6.3 Intelligent Soil

17

6.4 Intelligent Hydrogel

17

6.5 Sensor-Integrated Mulching

18

7. Crop Lifecycle and Environmental Intelligence

19

7.1 Why Lifecycle State Matters

19

7.2 Growing Degree Days

19

7.3 Stage-Aware Threshold Intelligence

19

7.4 Environmental Coupling

19

8. Machine Learning Methodology and Models

20

8.1 Role of ML in the Final Architecture

20

8.2 Model 1 — Grape Yield Prediction

20

8.3 Model 2 — Telemetry and Hydrogel Trigger

21

8.4 Model Selection and Limitations

21

9. Backend Engineering and Service Layer

22

9.1 Backend Responsibility

22

9.2 FastAPI Service Boundary

22

9.3 Input Validation and Separation of Concerns

22

9.4 Execution and Local Development

22

10. Persistence, SQLite and State Recovery

24

10.1 Why SQLite Was Selected

24

10.2 Telemetry Table

24

10.3 State Recovery Mechanism

24

10.4 Production Considerations

24

11. Digital Twin Validation and Backend Integration

25

11.1 Validation Philosophy

25

11.2 Causal Intervention Evidence

25

11.3 Integration Readiness

25

12. Frontend Engineering and Dashboard

26

12.1 Frontend Purpose

26

12.2 Final Dashboard Evidence

26

12.3 Panel-to-System Relationship

27

12.4 Frontend Design Principle

28

13. End-to-End Integration

29

13.1 Integrated System Flow

29

13.2 Canonical State Principle

29

13.3 Failure Boundaries 29

13.4 Current Integration Position 29

14. Technology Stack and Development Environment 30

14.1 Technology Stack 30

14.2 Development Practices 30

14.3 Reproducibility 30

15. Testing, Validation and Results 31

15.1 Test Strategy 31

15.2 Digital Twin Results 31

15.3 ML Results 31

15.4 Validation Summary 32

16. Security, Reliability and Operational Considerations 33

16.1 Why These Concerns Matter 33

16.2 Reliability Considerations 33

16.3 Security Boundary 33

16.4 Observability 33

17. Performance, Scalability and Deployment Readiness 34

17.1 Prototype Performance Position 34

17.2 Scalability Path 34

17.3 Deployment Readiness Criteria 34

17.4 Current Readiness Statement 34

18. Project Management, Team Contributions and Engineering Workflow 35

18.1 Work Distribution 35

18.2 Engineering Workflow 35

18.3 Iterative Review 35

18.4 Lessons from the Engineering Process 35

19. Limitations and Risk Analysis 36

19.1 Data Limitations 36

19.2 Model Limitations 36

19.3 Twin Calibration Limitations 36

19.4 Risk Register 36

20. Future Scope and Deployment Roadmap 37

20.1 Immediate Next Engineering Stage 37

20.2 Medium-Term Extensions 37

20.3 Multimodal Extension 37

20.4 Deployment Roadmap 37

21. Final Evaluation and Conclusion 38

21.1 Objective-wise Completion 38

21.2 Principal Technical Contributions 38

21.3 Conclusion 38

22. References 39

A. Technical Evidence Classification 40

A.1 Evidence Boundary 40

B. Representative State and Data Schema 41

B.1 Temporal Semantics 41

C. Digital Twin Validation Evidence 42

D. Frontend Evidence Register 43

E. Model Evaluation Evidence 47

F. Backend and Persistence Evidence 48

G. Final Frontend Gallery 49

H. Deployment Readiness Checklist 52

1. Introduction and Project Context

1.1 Project Background

Agricultural decision-making is affected by interacting variables rather than isolated measurements. Soil condition, weather, crop identity, growth stage, irrigation behaviour and interventions influence one another over time. A useful digital agricultural system must therefore preserve context and temporal continuity while remaining understandable to the farmer or agronomist.

Apollo AgriVerse — AgriIntel was developed as a focused intelligence package within the broader Apollo AgriVerse ecosystem. The final engineering target is an explainable crop and field intelligence loop: acquire evidence, represent the evolving farm condition, evaluate it with rules and predictive models, persist the result, and present a decision-relevant interpretation.

1.2 Problem Definition

The initial problem framing emphasized prediction and dashboard presentation. During review, that framing was found to be incomplete because an agricultural recommendation cannot be separated from the condition in which it is produced. The system therefore had to account for soil and region, crop and variety, environmental forcing, intervention state and lifecycle progression.

Problem dimension	Engineering implication
Static crop recommendation	Requires soil, region and crop-context representation.
Instantaneous sensor value	Requires temporal state and history.
Prediction without context	Requires canonical state and feature lineage.
Dashboard-only architecture	Requires backend orchestration and persistence.
Synthetic simulation without evidence boundary	Requires explicit classification of simulated vs observed evidence.

Table 1.1 — Problem dimensions identified during project maturation.

1.3 Final Engineering Objective

The final objective is not to claim a commissioned autonomous farm. It is to implement and demonstrate a technically coherent AgriIntel package in which agricultural evidence can be transformed into a persistent, evolving virtual state, enriched by predictive intelligence, exposed through controlled services and rendered through a functional frontend.

2. Requirements, Scope and Project Evolution

2.1 Requirements Maturation

The project evolved through iterative technical review. The most consequential refinement was recognizing that the solution needed to represent the agricultural system rather than merely calculate isolated variables. This led to soil and regional context, crop and variety mapping, lifecycle tracking, intervention modelling and a stronger separation between state, intelligence and presentation.

- Soil type and regional variation were promoted to first-class agricultural inputs.
- Crop and variety suitability became part of the context used by the intelligence layer.
- Crop lifecycle became a dynamic state rather than a static label.
- Intelligent soil, hydrogel and mulching concepts were represented as interacting state components.
- India-wide soil and weather sources were considered to improve geographic coverage.
- The digital twin was defined as a computational state-and-transition system, not simply a visual 3D model.

2.2 Functional Requirements

ID	Requirement	Final implementation direction
FR-01	Represent farm context	Farm identity, crop and environmental state.
FR-02	Track temporal state	State transition and persistence engine.
FR-03	Represent interventions	Hydrogel and mulching state variables.
FR-04	Track crop lifecycle	GDD-driven phenology state.
FR-05	Generate predictions	Yield and telemetry ML pipelines.
FR-06	Expose intelligence	Backend API boundary.
FR-07	Present outputs	Multi-panel dashboard interface.
FR-08	Preserve evidence	Database history, artifacts and validation records.

Table 2.1 — Final functional requirements and corresponding engineering implementation.

2.3 Non-Functional Requirements

- Traceability: important outputs should be explainable from inputs, state and model context.
- Reproducibility: simulations and model experiments should be rerunnable from retained artifacts.
- Modularity: data, state, ML, backend and frontend concerns should remain separable.
- Reviewability: the system should expose enough evidence for academic and engineering evaluation.
- Extensibility: future real telemetry, image intelligence and production infrastructure should fit without rewriting the conceptual architecture.

3. System Scope and Solution Strategy

3.1 AgriIntel Work Package

AgriIntel is treated as the agricultural intelligence vertical within Apollo AgriVerse. Its boundary includes agricultural data engineering, digital-twin state representation, intervention and lifecycle simulation, predictive ML, persistence, backend orchestration and the user-facing dashboard. Hardware fabrication and commissioned field deployment remain outside the evidence represented in this report.

3.2 Solution Strategy

Reviewable Agricultural Decision Loop

The end-to-end objective is a traceable loop rather than a collection of disconnected screens.

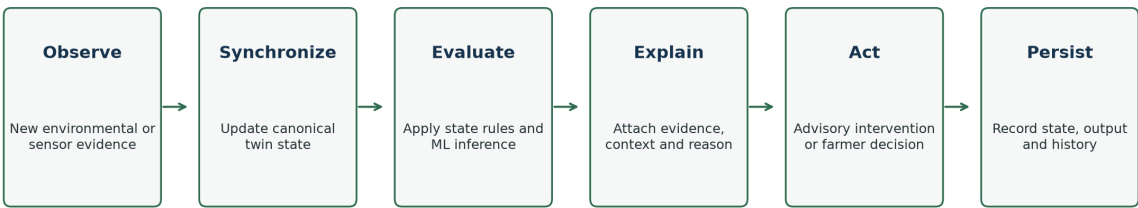


Figure 3.1 — The reviewable decision loop that connects evidence, state, intelligence, action and persistence.

The solution strategy is deliberately layered. The data layer prepares evidence; the state layer maintains temporal continuity; the intelligence layer adds learned or rule-based interpretation; the backend controls access to these capabilities; and the frontend translates outputs into an understandable interface. This decomposition prevents each dashboard panel from becoming an independent source of business logic.

3.3 Evidence Classes

Evidence class	Meaning in this project	Example
Observed / collected	Evidence originating from available agricultural sources or actual project inputs.	Public soil/weather/crop records.
Implemented	Executable project capability demonstrated by retained code or working subsystem.	State engine, persistence, dashboard panel.
Evaluated	Output measured under a documented test or notebook evaluation protocol.	ML test metrics; twin validation tests.
Simulated	Controlled synthetic or rule-generated trajectory.	Hydrogel intervention scenario.
Proposed	Architecture or capability planned beyond current evidence.	Continuous real-IoT learning loop.

Table 3.1 — Evidence taxonomy used to prevent overstatement in the final report.

4. Data Acquisition and Engineering

4.1 Data Strategy

The final data foundation combines heterogeneous agricultural sources with controlled synthetic data. The distinction is important: real or public datasets provide observed relationships and domain context, while synthetic streams provide deterministic coverage for state transitions, intervention scenarios, integration tests and edge cases that are difficult to obtain from limited field observations.

Data Engineering and Integration Pipeline

Real and synthetic evidence are kept conceptually distinct while remaining interoperable.

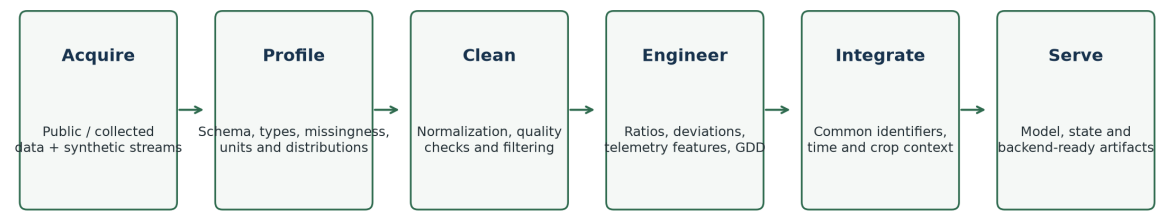


Figure 4.1 — Data acquisition, profiling, transformation, feature engineering and service preparation pipeline.

4.2 Dataset Inventory

Dataset / component	Role	Representative variables	Use in final system
Soil	Agronomic context	pH, N, P, K, texture, moisture indicators	State initialization, suitability and intelligence
Weather	Environmental forcing	Temperature, humidity, rainfall and related climate fields	State transitions and lifecycle
Crop	Crop context	Crop identity, requirements and mappings	Recommendation context
Farm metadata	Twin identity	Farm ID, location/context, configuration	System identity
Sensor stream	Dynamic telemetry	Moisture, temperature, humidity and related signals	State synchronization
Hydrogel	Intervention state	Capacity / release-related variables	Intervention behaviour
Mulching	Surface intervention	Degradation and cooling-related variables	Surface protection state
Crop lifecycle	Phenology	GDD, stage and transitions	Lifecycle state

Table 4.1 — Data components and their role in the AgriIntel architecture.

4.3 Data Cleaning and Harmonization

The engineering pipeline requires schema alignment, type normalization, missing-value treatment, unit consistency, temporal ordering and geographical consistency. Feature engineering is performed after the underlying semantics are understood; for example, ratios and deviations can be useful to ML but should not obscure the physical meaning of the original variables.

4.4 Synthetic Data Generation

Synthetic generation is used as a controlled engineering instrument. A synthetic stream can deliberately drive moisture downward, introduce rainfall, activate hydrogel release, degrade mulch or advance lifecycle state. The purpose is to test causality and integration behaviour, not to represent synthetic observations as field measurements.

5. Digital Twin Architecture

5.1 Digital Twin Definition for AgriIntel

For this project, a digital twin is a computational representation that maintains meaningful correspondence with a physical or simulated agricultural system over time. It is therefore more than a dashboard and more than a 3D visualization. Its essential property is state continuity: the current condition depends on prior condition and incoming evidence.

Apollo AgriVerse — AgriIntel System Architecture

The system separates evidence, state, intelligence, service orchestration and presentation.

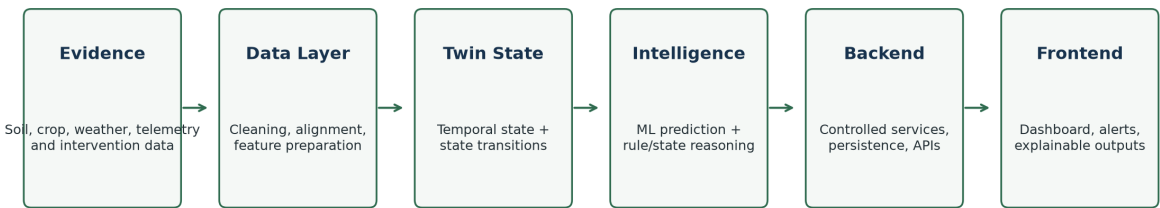


Figure 5.1 — Layered AgriIntel architecture from agricultural evidence to state, intelligence, services and presentation.

5.2 Twin State Vector

The state vector contains the variables required to describe the evolving farm condition. The documented representation includes farm identity, soil condition, environmental forcing, intervention condition, crop lifecycle and time. Derived intelligence can be associated with the state where required for traceability.

State group	Representative fields	Purpose
Farm	farm_id, region/context	Identifies the virtual farm and its geographic/agronomic context.
Soil	pH, N, P, K, moisture	Represents substrate condition and water availability.
Environment	temperature, humidity, rainfall	Provides external forcing for transitions.
Intervention	hydrogel capacity, mulch degradation	Represents active agricultural interventions.
Crop	crop_id, variety, stage, cumulative GDD	Represents biological lifecycle context.
Time	timestamp / simulation step	Maintains temporal ordering and replayability.
Intelligence	prediction, risk, recommendation context	Associates derived outputs with their state context.

Table 5.1 — Representative digital-twin state schema.

5.3 State Transition Principle

$$S(t+1) = \text{Clip}(S(t) - \text{Evapotranspiration}(t) + \text{Rainfall}(t) + \text{HydrogelRelease}(t), 10.0, 90.0)$$

The transition equation gives the simulator a deterministic causal structure. Moisture is not regenerated randomly at every timestep; the next state depends on the current state and explicit forcing/intervention terms. This makes scenario comparison and validation possible.

6. State Models and Agricultural Simulation

6.1 Why State Models Were Required

Several AgriIntel variables have memory. Soil moisture today depends on previous moisture and water loss/addition. Crop lifecycle progresses rather than changing arbitrarily. Hydrogel capacity changes through absorption and release. Mulching changes as the protective layer degrades. A state model makes these dependencies explicit and auditable.

State Model Abstraction

State models provide explicit temporal memory; ML models provide learned relationships where appropriate.



Figure 6.1 — State-transition abstraction used to represent evolving agricultural conditions.

6.2 Alternatives Considered

Approach	Strength	Limitation for AgriIntel	Decision
Static dataset	Simple and easy to query	No evolving condition or temporal memory	Not sufficient as primary state representation
Pure ML	Can learn nonlinear relationships	Does not inherently define an auditable physical state	Used selectively for prediction, not as twin state
Fully mechanistic model	Strong interpretability	Can require detailed physical parameters and calibration	Too rigid for the current prototype scope
State-transition model	Explicit temporal continuity and controllable scenarios	Requires carefully defined transitions and calibration	Selected as core twin representation
Hybrid state + ML	Combines continuity with learned relationships	More integration complexity	Selected as final architectural direction

Table 6.1 — Architecture alternatives considered for representing the evolving farm.

6.3 Intelligent Soil

The intelligent-soil representation treats soil as an evolving medium rather than a static crop attribute. Moisture is the primary dynamic variable in the current twin implementation, while NPK and thermal variables provide the wider agricultural context. The design leaves room for richer nutrient and root-zone models once calibrated field observations become available.

6.4 Intelligent Hydrogel

The hydrogel component represents water-retention intervention behaviour. Its role is not simply to store a percentage value; it changes the moisture trajectory by contributing a release term when intervention conditions are satisfied. This allows the twin to answer a more meaningful question: how does the field state change if the intervention is applied?

6.5 Sensor-Integrated Mulching

The mulching state represents the protective surface layer and its degradation over time. The architecture associates mulch condition with surface thermal and evaporation-related effects. The current implementation is simulation-oriented; physical material calibration remains future validation work.

7. Crop Lifecycle and Environmental Intelligence

7.1 Why Lifecycle State Matters

A crop does not have a constant water or environmental requirement throughout its lifecycle. A digital twin that ignores lifecycle stage can produce internally consistent but agriculturally inappropriate decisions. AgriIntel therefore represents grape phenology as a dynamic state influenced by accumulated heat.

7.2 Growing Degree Days

$$GDD = ((T_{max} + T_{min}) / 2) - T_{base}$$

The documented grape implementation uses a base temperature of 10°C. A day with T_{max} = 28°C and T_{min} = 16°C produces an average temperature of 22°C and therefore 12 GDD units. Cumulative GDD can then advance the virtual crop through the defined phenological stages.

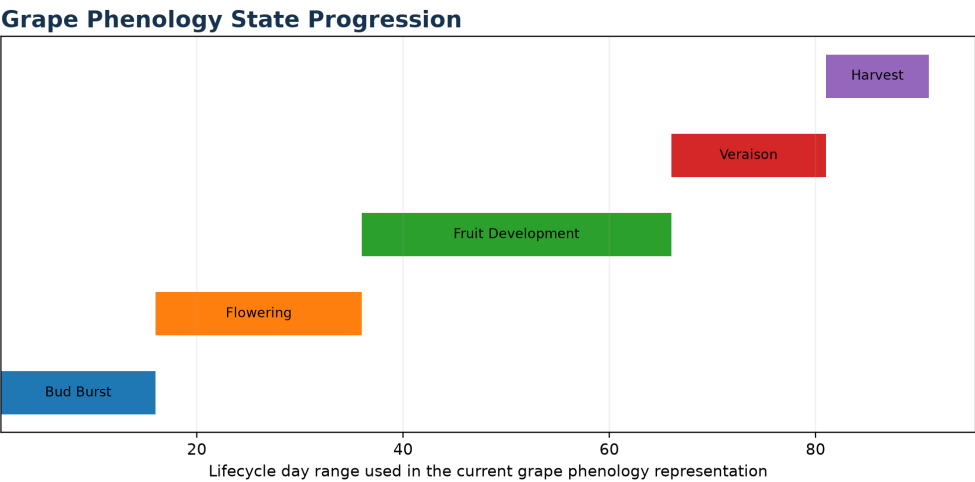


Figure 7.1 — Current grape phenology stage ranges represented in the digital-twin lifecycle model.

7.3 Stage-Aware Threshold Intelligence

Stage	Day range	Evaporation multiplier	Moisture threshold	Interpretation
Bud Burst	1–15	0.45×	25%	Initial root awakening; moderate water need.
Flowering	16–35	0.85×	35%	High sensitivity to water stress.
Fruit Development	36–65	1.10×	30%	Peak canopy transpiration and cell expansion.
Veraison	66–80	0.70×	22%	Controlled deficit may support maturity behaviour.
Harvest	81–90+	0.35×	18%	Lower water requirement near harvest.

Table 7.1 — Documented stage-aware grape phenology thresholds.

7.4 Environmental Coupling

Weather acts as external forcing for the twin. Temperature affects lifecycle accumulation and water demand; rainfall contributes water; humidity and related variables provide context for stress and telemetry intelligence. The important design principle is that environmental data are consumed as inputs to state evolution rather than displayed independently of the virtual farm condition.

8. Machine Learning Methodology and Models

8.1 Role of ML in the Final Architecture

Machine learning is positioned as predictive intelligence, not as a replacement for the digital-twin state. The state engine answers “what is the current modeled condition and how does it evolve?” ML answers questions such as “what learned relationship can estimate an outcome from the available features?” Keeping the responsibilities separate makes the architecture easier to test and explain.

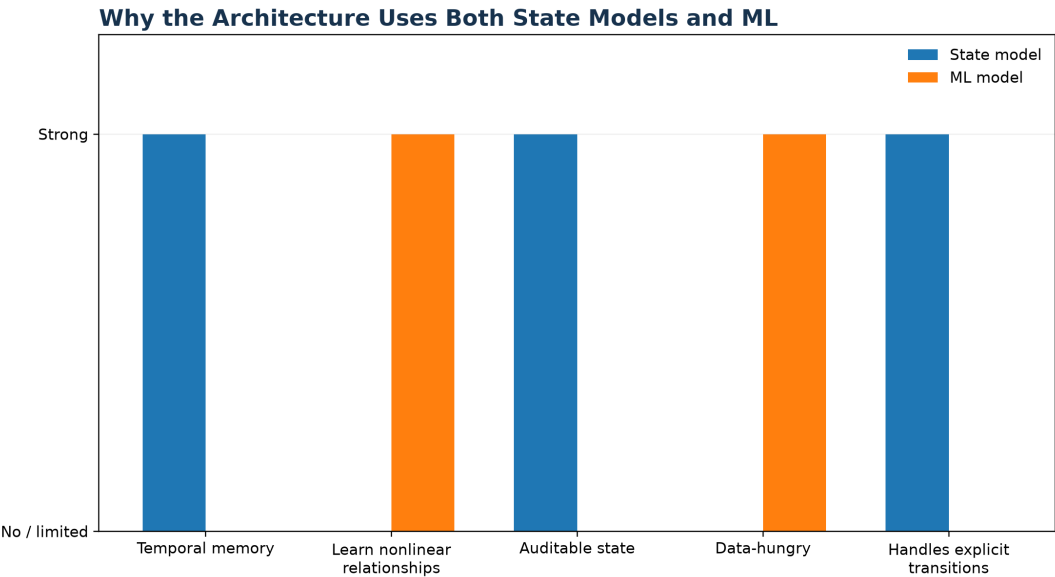


Figure 8.1 — Complementary responsibilities of state models and machine-learning models.

8.2 Model 1 — Grape Yield Prediction

The first ML pipeline uses an 800-row, 9-column grape agronomy dataset. Random Forest and Gradient Boosting regressors were compared using 5-fold cross-validation. The retained baseline was Random Forest based on the comparative experiment, but the held-out performance does not justify a production-readiness claim.

Metric / artifact	Value
Target	yield_tons_ha
Dataset	800 rows; 9 columns
Split	640 training / 160 test
Validation	5-fold cross-validation
CV R ²	-0.0944 ± 0.0299
Test RMSE	0.6134 tons/ha
Test MAE	0.4569 tons/ha
Test R ²	-0.0942
Interpretation	Baseline retained for comparison; not production-ready.

Table 8.1 — Grape-yield model evaluation evidence.

8.3 Model 2 — Telemetry and Hydrogel Trigger

The second pipeline uses a 1,000-row, 24-column digital-twin telemetry dataset to predict hydrogel water-storage percentage. The documented model is Gradient Boosting with 150 estimators, learning rate 0.08 and maximum depth 4. Engineered features include thermal gap, effective mulch cooling and an evapotranspiration-loss index.

Metric / artifact	Value
Target	hydrogel_water_storage_pct
Dataset	1,000 rows; 24 columns
Split	800 training / 200 test
Model	Gradient Boosting Regressor
Test RMSE	0.9484%
Test MAE	0.1456%
Test R ²	0.9900
Interpretation	Strong notebook-level result; broader validation required.

Table 8.2 — Telemetry hydrogel-trigger model evaluation evidence.

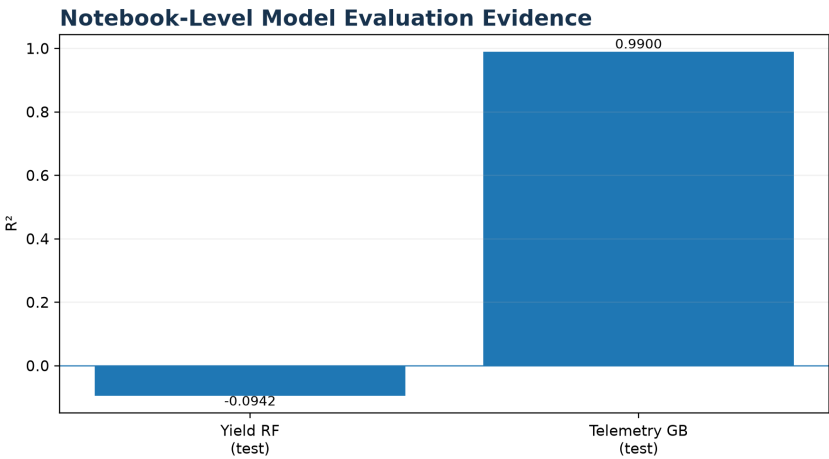


Figure 8.2 — Comparison of the documented held-out R^2 values; negative yield-model R^2 is intentionally shown rather than hidden.

8.4 Model Selection and Limitations

The final ML position is therefore deliberately asymmetric. The telemetry trigger model provides a strong computational result on its defined test split, while the yield model remains a weak baseline that should not be presented as accurate production intelligence. This is an important engineering conclusion: model selection is based on evidence, not on the presence of an ML algorithm.

Neither model has been field-validated through a commissioned agricultural sensor deployment. The reported metrics describe the retained notebook evaluation conditions and should be interpreted within those boundaries.

9. Backend Engineering and Service Layer

9.1 Backend Responsibility

The backend is the controlled integration boundary between the frontend and the computational services. Its role is to accept validated inputs, retrieve or update canonical state, invoke the appropriate intelligence service and return structured outputs. It should not duplicate state-transition equations or independently reconstruct model logic in every HTTP route.

Backend Integration Boundary

The backend orchestrates services; it should not duplicate the domain logic owned by the twin or model layer.

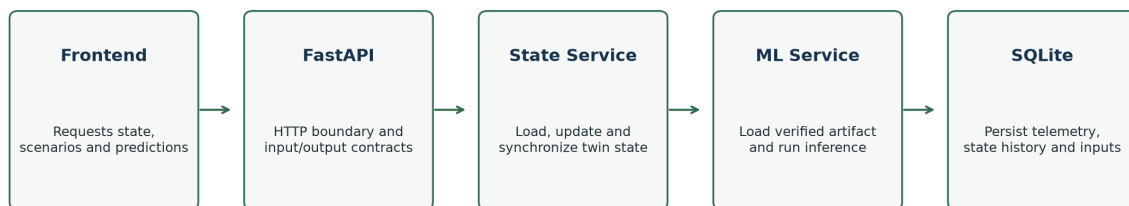


Figure 9.1 — Backend integration boundary between frontend requests, state services, ML inference and persistence.

9.2 FastAPI Service Boundary

The documented implementation uses FastAPI as the REST service layer. The technical guide identifies prediction routes for grape yield and telemetry/hydrogel-trigger inference, together with twin-state and scenario-oriented service behaviour. The API boundary allows the frontend to consume the system without embedding the computational implementation inside the browser.

Service category	Input	Processing responsibility	Output
Yield prediction	Soil/agronomy feature payload	Feature transformation + retained model inference	Expected yield estimate
Telemetry trigger	Telemetry / microclimate payload	Feature engineering + retained trigger model	Hydrogel storage requirement
Twin state	Farm/state request	Retrieve current canonical state	Current virtual farm state
Scenario execution	Scenario parameters	Run controlled state evolution	Scenario trajectory / result
Dashboard data	Panel-specific request	Aggregate canonical outputs	UI-ready structured data

Table 9.1 — Backend service categories and their responsibilities.

9.3 Input Validation and Separation of Concerns

A production-oriented service boundary should reject malformed inputs before they reach model or state logic. The current architecture therefore treats schema validation, state logic, ML inference and presentation as distinct concerns. This is also important for future deployment because it prevents frontend assumptions from becoming hidden model or database dependencies.

9.4 Execution and Local Development

```
uvicorn main:app --reload --port 8000
```

The documented development workflow launches the FastAPI application locally. The end-term report treats this as an engineering execution path, not as evidence of production hosting, authentication hardening, distributed scaling or field deployment.

10. Persistence, SQLite and State Recovery

10.1 Why SQLite Was Selected

SQLite was selected as the prototype persistence layer because the digital-twin workload benefits from local, low-overhead reads and writes, minimal configuration and offline operation. The database can live alongside the prototype and remain directly inspectable during development and testing. The choice is not presented as the final production database architecture.

10.2 Telemetry Table

Column	Type	Meaning
day	INTEGER	Sequential simulation day index.
stage	TEXT	Active grape phenology stage.
soil_moisture	REAL	Current soil moisture state (%).
evapotranspiration	REAL	Daily atmospheric moisture loss contribution.
rainfall	REAL	Natural rainfall contribution.
hydrogel_release	REAL	Hydrogel intervention contribution.

Table 10.1 — Documented twin_telemetry persistence schema.

SQLite Persistence Model

The documented recovery path demonstrates why persistence is part of the digital-twin implementation, not an optional add-on.

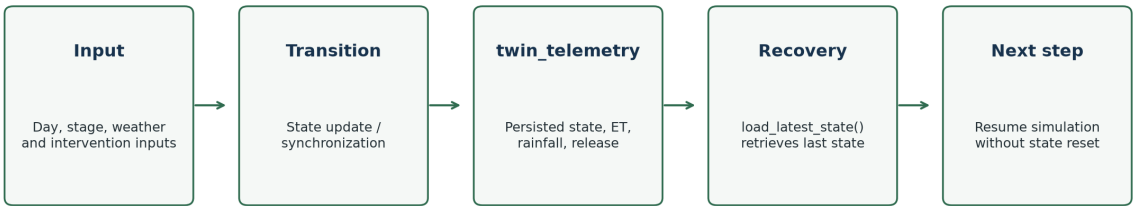


Figure 10.1 — Persistence and recovery flow used by the digital-twin backend.

10.3 State Recovery Mechanism

The documented recovery function `load_latest_state()` queries the most recent row from `twin_telemetry`. In the recorded validation scenario, the simulation was run through Day 30, the process was interrupted, and the persisted Day 30 state was recovered before calculating Day 31. This demonstrates that persistence is functionally connected to state continuity rather than being a passive data dump.

```
SELECT * FROM twin_telemetry ORDER BY day DESC LIMIT 1
```

10.4 Production Considerations

A production deployment would need stronger concurrency guarantees, backup strategy, migration management, observability and potentially a server-grade database. The current SQLite implementation is therefore best understood as a transparent and reviewable persistence layer for the internship prototype.

11. Digital Twin Validation and Backend Integration

11.1 Validation Philosophy

Validation was organized around properties of the system rather than around a single numerical score. A useful twin must preserve state continuity, respond causally to interventions, support alternative scenarios, recover from persistence and change lifecycle-aware behaviour when the stage changes.

Test	Condition / method	Observed result	Status
State continuity	Run Day 1 → 90 step-wise	Continuous trajectory; ending moisture 22.6%	PASSED
Causal intervention	Low moisture + hydrogel intervention	Moisture changed from 19.9% without action to 27.8% with intervention	PASSED
Branching logic	Drought / hydrogel / rainfall scenarios	Distinct state trajectories	PASSED
Database recovery	Run Day 1–30 → purge/process interruption → resume	Day 30 state restored; Day 31 calculated	PASSED
Stage intelligence	Evaluate phenology thresholds	Hydrogel bounds adjust with stage	PASSED

Table 11.1 — Documented digital-twin integration validation suite.

11.2 Causal Intervention Evidence

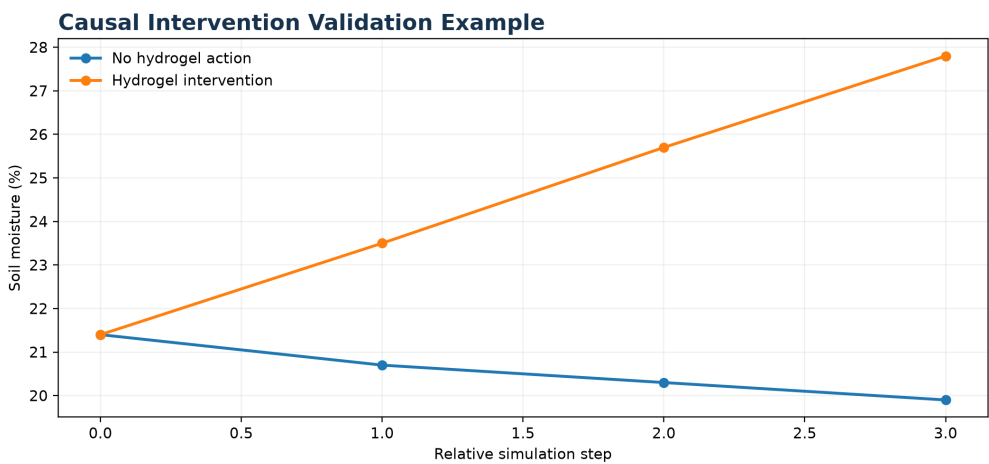


Figure 11.1 — Illustrative rendering of the documented causal-intervention validation values.

Evidence boundary. The intervention plot is based on documented simulator validation values. It demonstrates modelled causality inside the simulation and does not establish field-level hydrogel efficacy.

11.3 Integration Readiness

The validation suite establishes that the twin engine can be tested independently before being exposed through the backend. This layered approach reduces debugging ambiguity: a failure in an API response can be distinguished from a state-transition defect, and a model-output problem can be distinguished from a persistence problem.

12. Frontend Engineering and Dashboard

12.1 Frontend Purpose

The frontend is the communication surface of AgriIntel. Its purpose is not to display as many charts as possible, but to present the state of the agricultural system, model outputs and intervention context in a way that a user can interpret. The final interface therefore groups the product around operational views such as the digital twin, soil, hydrogel, lifecycle, weather, analytics, predictions, simulation and alerts.

12.2 Final Dashboard Evidence

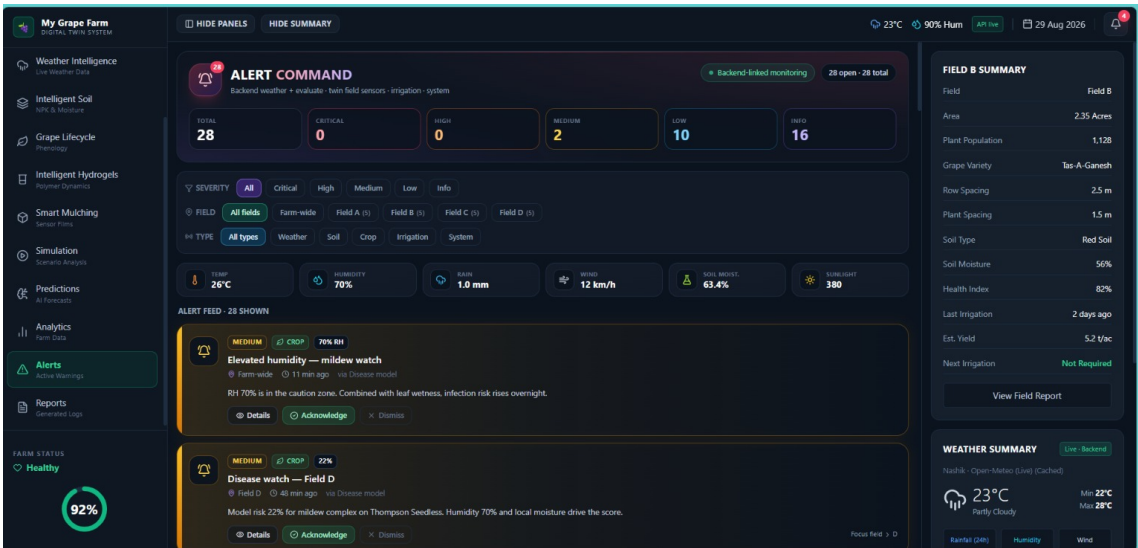


Figure 12.1 — Alert Panel — representative final frontend evidence.

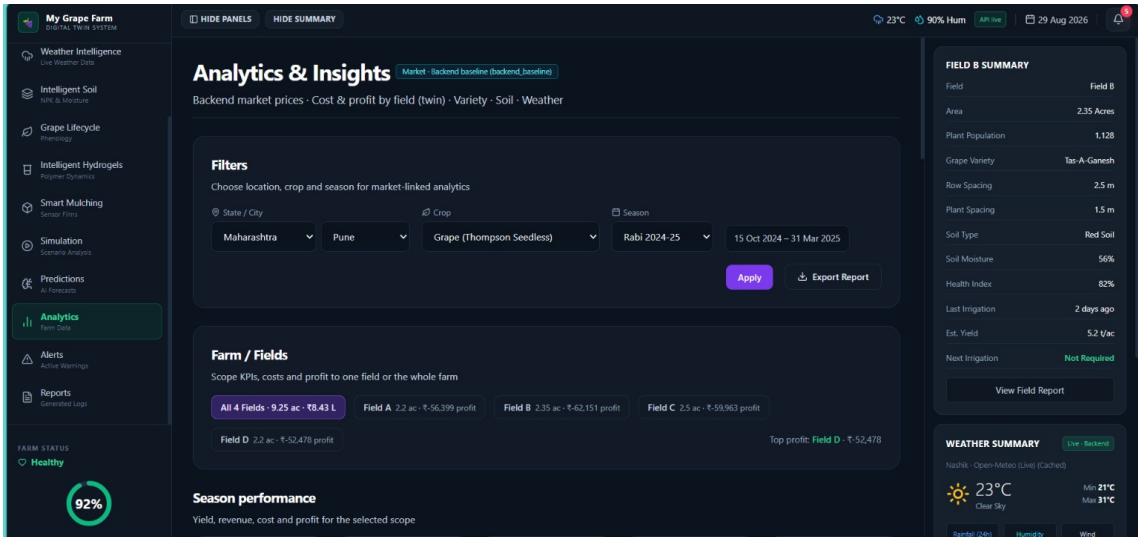


Figure 12.2 — Analytics Panel — representative final frontend evidence.

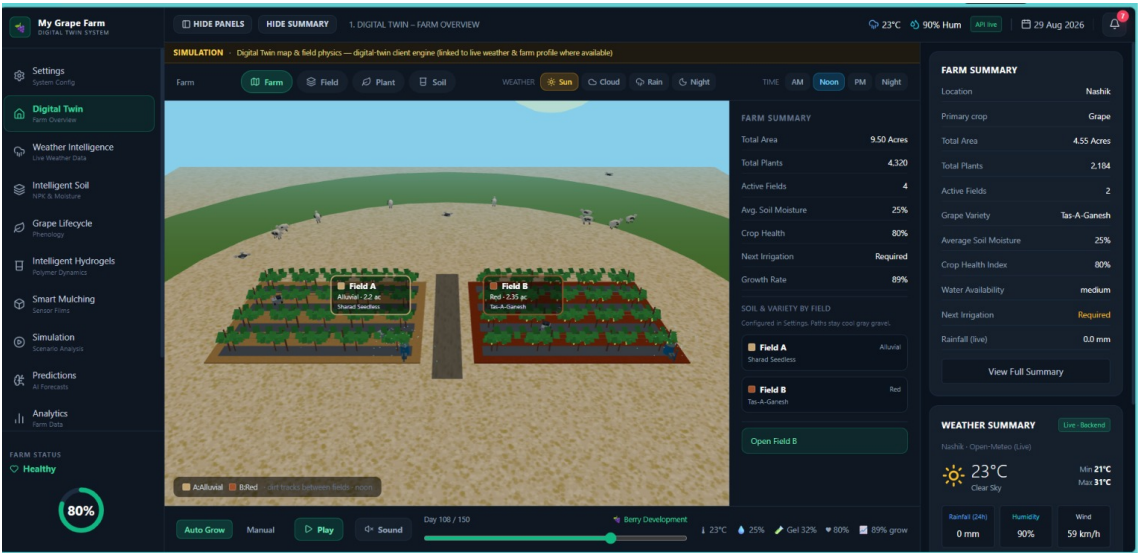


Figure 12.3 — Digital Twin Panel — representative final frontend evidence.

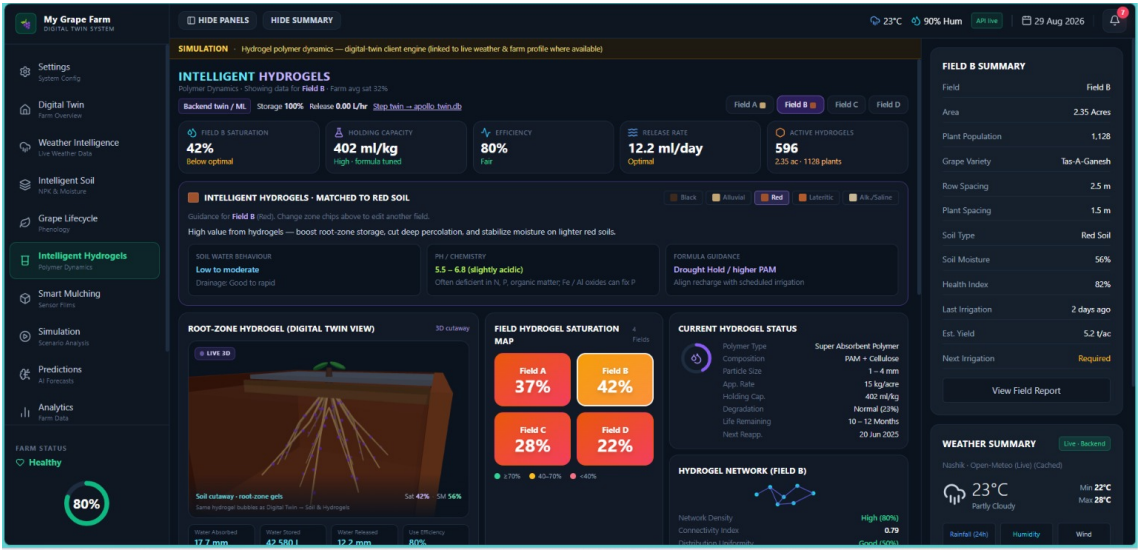


Figure 12.4 — Hydrogel Panel — representative final frontend evidence.

The screenshots above are treated as frontend evidence. They demonstrate interface implementation and visual integration; they do not, by themselves, prove that every displayed value is connected to a live field sensor or production backend.

12.3 Panel-to-System Relationship

Panel family	Expected source	Decision context
Digital Twin	Canonical twin state	Current virtual farm condition
Intelligent Soil	Soil data + state	Substrate condition and needs
Hydrogel	State + trigger model	Water-retention intervention status
Smart Mulching	Mulch state	Surface protection / degradation
Lifecycle	GDD + crop state	Growth stage and progression
Weather	Weather dataset / service	Environmental forcing
Prediction	ML inference	Expected outcome / estimate
Analytics	Aggregated system outputs	Trend and comparative interpretation
Alerts	Thresholds + intelligence	Action-oriented warning

Table 12.1 — Relationship between representative dashboard panels and computational sources.

12.4 Frontend Design Principle

The intended production principle is that dashboard panels should consume canonical backend outputs rather than independently recreating state or model logic. This avoids contradictory values across panels and makes the interface a presentation layer over the same computational truth.

13. End-to-End Integration

13.1 Integrated System Flow

Evidence Lineage from Input to Dashboard

A final system claim should be traceable to the evidence class and artifact that produced it.

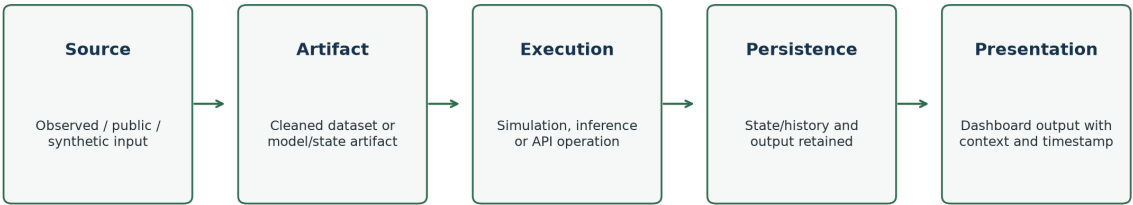


Figure 13.1 — Evidence lineage from agricultural input through computation, persistence and dashboard presentation.

The end-to-end package can be understood as a sequence of transformations. Agricultural data and telemetry become normalized computational inputs. The twin synchronizer turns those inputs into an evolving state. ML services consume controlled feature representations to produce predictions. The backend provides the service boundary and persistence records the relevant state and outputs. The frontend renders the result in operational context.

13.2 Canonical State Principle

The canonical-state principle is central to integration. There should be one authoritative representation of the current virtual farm condition. Model inference should consume that state or controlled derivatives of it, and the frontend should render the resulting outputs rather than maintain a parallel state machine.

13.3 Failure Boundaries

Failure location	Typical symptom	Isolation strategy
Data layer	Malformed or inconsistent feature input	Schema/profile checks before service execution
State layer	Unexpected trajectory	Run deterministic twin validation suite
ML layer	Invalid / unstable prediction	Check model artifact, feature order and evaluation evidence
Persistence	State resets or history missing	Verify database write and recovery query
Backend	API error or wrong payload	Validate service contract independently
Frontend	Contradictory or stale panel	Compare UI payload with canonical backend state

Table 13.1 — Failure isolation boundaries in the integrated architecture.

13.4 Current Integration Position

The architecture is substantially integrated at the prototype level: state engines, persistence, ML artifacts, FastAPI service boundaries and frontend views have defined roles. The remaining distinction is between subsystem integration and a fully field-connected production loop. The latter requires real telemetry, broader validation, deployment hardening and operational monitoring.

14. Technology Stack and Development Environment

14.1 Technology Stack

Layer	Technology / tool	Purpose
Programming	Python	Data processing, simulation, orchestration and ML.
Data engineering	Pandas / NumPy	Cleaning, transformation and feature preparation.
ML	Scikit-learn	Regression models and evaluation.
Simulation	Python state engines	Temporal state transitions and synthetic streams.
Persistence	SQLite	Local twin state/history storage.
Backend	FastAPI / Uvicorn	REST service and local application server.
Frontend	HTML / JavaScript / Tailwind-based dashboard	User-facing visualization and interaction.
Version control	Git / GitHub	Source control and collaborative engineering.
Documentation	ReportLab / generated technical documents	Reproducible project documentation.

Table 14.1 — Technology stack used across the final AgriIntel prototype.

14.2 Development Practices

- Modular separation of datasets, state logic, models, backend and frontend.
- Retained model artifacts and simulation outputs where applicable.
- Validation scripts for the digital-twin core before backend exposure.
- Automated report generation for consistent technical documentation.
- Git-based version control for project evolution and recovery.

14.3 Reproducibility

Reproducibility is treated as an engineering requirement. The final report identifies the relevant model notebooks, state validation suite, persistence schema and generated evidence. A future production-grade evidence system should extend this with immutable dataset checksums, environment manifests, model hashes, API contract versions and execution run identifiers.

15. Testing, Validation and Results

15.1 Test Strategy

Testing is divided into data quality checks, model evaluation, state-engine validation, persistence recovery, API integration and frontend verification. This decomposition is intentional: a single end-to-end test cannot explain whether a failure originates in the data, model, state transition, service boundary or interface.

15.2 Digital Twin Results

- State continuity test passed for a Day 1 → Day 90 run.
- Causal intervention test showed a materially different moisture trajectory under hydrogel intervention.
- Scenario branching produced distinct drought, intervention and rainfall trajectories.
- Database recovery restored the latest persisted state and continued the simulation.
- Phenology-stage intelligence changed thresholds according to lifecycle state.

15.3 ML Results

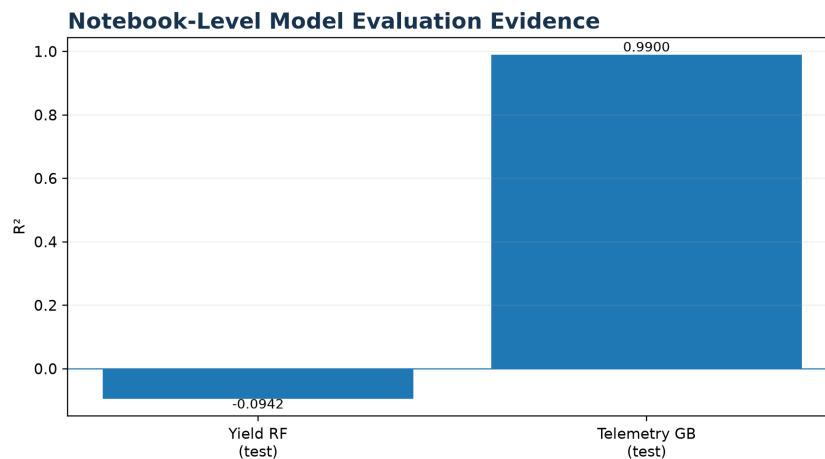


Figure 15.1 — Held-out R^2 values from the two documented ML pipelines.

The ML results show why numerical evaluation must be interpreted rather than merely reported. The yield model's negative R^2 indicates poor held-out performance under the documented split. The telemetry model's R^2 of 0.9900 indicates a strong fit on its test data, but the dataset is synthetic/digital-twin telemetry and therefore does not establish real-world generalization by itself.

15.4 Validation Summary

Subsystem	Evidence	Current conclusion
Data engineering	Prepared heterogeneous and synthetic components	Implemented pipeline; broader source validation continues
State engine	Five documented validation tests	Core state behaviour demonstrated
Persistence	SQLite recovery scenario	State continuity across restart demonstrated
Yield ML	Cross-validation + held-out metrics	Baseline retained; weak performance
Telemetry ML	Held-out test metrics	Strong computational result; needs broader validation
Backend	FastAPI execution and documented service boundary	Prototype service layer implemented
Frontend	Final dashboard panels	Functional prototype evidence
Field deployment	No commissioned sensor deployment evidence	Not claimed

Table 15.1 — Subsystem-level validation and current conclusions.

16. Security, Reliability and Operational Considerations

16.1 Why These Concerns Matter

A research prototype can operate with simplified infrastructure, but an industry-grade architecture must identify operational risks before deployment. Agricultural recommendations affect real decisions, so incorrect state, stale telemetry, invalid model inputs or untraceable outputs can be more damaging than an ordinary UI defect.

16.2 Reliability Considerations

Concern	Current approach	Production extension
State loss	SQLite persistence + recovery query	Managed database + backups + transaction monitoring
Invalid input	Service-layer validation direction	Strict schemas, validation and rejection logging
Model drift	Notebook-level evaluation	Scheduled evaluation and drift monitoring
Stale telemetry	Temporal state representation	Timestamp freshness and sensor-health checks
Contradictory UI	Canonical backend state principle	Contract tests and frontend integration tests
Traceability	Retained state/model evidence	Run IDs, artifact hashes and audit logs

Table 16.1 — Reliability and operational considerations for production maturation.

16.3 Security Boundary

The internship implementation does not claim production-grade identity management, role-based authorization, secret management or network hardening. These belong to the deployment layer. The architecture nevertheless establishes an appropriate boundary because the frontend does not need direct access to the model files or database; it should communicate through controlled backend services.

16.4 Observability

A mature deployment should log input timestamps, state versions, model versions, inference outcomes, warnings and failures. Observability is especially important for a digital twin because the user needs to know whether a visible value represents current telemetry, simulated state, a prediction or a stale persisted observation.

17. Performance, Scalability and Deployment Readiness

17.1 Prototype Performance Position

The current architecture is optimized for transparent local development and review rather than distributed scale. SQLite provides low-friction persistence, Python keeps the simulation and ML environment cohesive, and FastAPI provides a lightweight service boundary. These choices minimize infrastructure complexity while the core engineering abstractions are being validated.

17.2 Scalability Path

Layer	Prototype choice	Scale-out path
Data	Local processed files / synthetic streams	Managed object storage + governed ingestion pipeline
State	Python state engine	Serviceized state workers / event-driven synchronization
Database	SQLite	PostgreSQL or equivalent managed relational store
ML	Local serialized artifacts	Model registry + inference service
Backend	FastAPI	Containerized API + horizontal scaling
Frontend	Browser dashboard	Static hosting + CDN + monitored API
Telemetry	Synthetic / prototype stream	MQTT/HTTP/edge gateway + device registry

Table 17.1 — Prototype-to-production scalability path.

17.3 Deployment Readiness Criteria

- Data integrity: validated schemas, provenance and reproducible preprocessing.
- State integrity: deterministic transitions and persisted history.
- Model integrity: versioned artifact, evaluation protocol and retained metrics.
- API integrity: controlled contracts, validation and error handling.
- UI integrity: panels display canonical outputs without duplicated computation.
- Decision integrity: recommendations can be traced to state, model and evidence.
- Operational integrity: deployment environment, logging, monitoring and failure behaviour verified.

17.4 Current Readiness Statement

The final project should be described as a technically integrated prototype and digital-twin-ready agricultural intelligence platform, not as a fully commissioned field product. This distinction is a strength of the report because it identifies the exact evidence needed for the next engineering stage.

18. Project Management, Team Contributions and Engineering Workflow

18.1 Work Distribution

Team member	Primary responsibility	Representative outputs
Vijayalaxmi K. Sundalam	Technical integration	Digital-twin architecture, state synchronization, backend/persistence integration and technical documentation.
Nandini N. Naral	Product, data quality & mining	Data collection, dataset analysis, product context and data-quality work.
Sunaina S. Gaikwad	ML baseline & evaluation	Model experimentation, comparison and evaluation evidence.
Dakshini A. Neel	Data & ML-model frontend	Frontend visualization and model-output presentation.

Table 18.1 — Primary team responsibilities and representative contributions.

18.2 Engineering Workflow

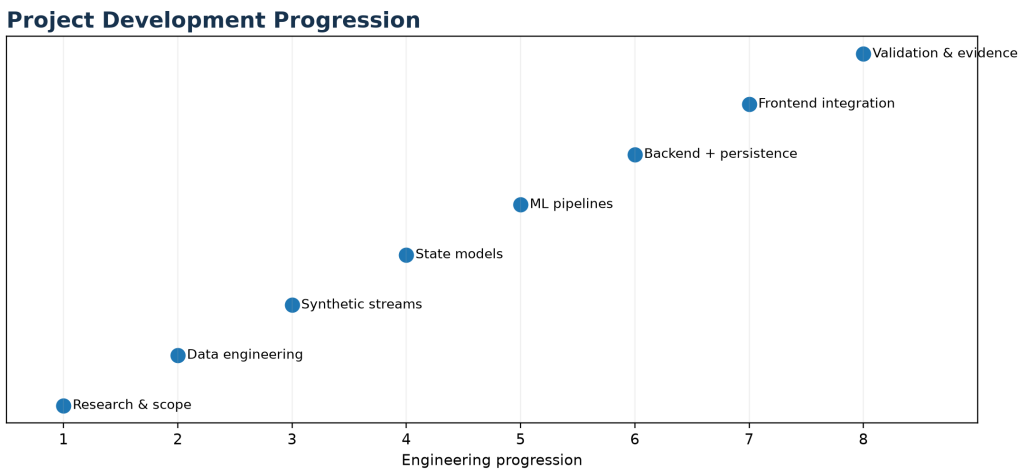


Figure 18.1 — Progression from research and data preparation through modelling, integration and validation.

18.3 Iterative Review

The project followed an iterative rather than strictly linear process. Research informed the first architecture; implementation exposed gaps; review introduced domain requirements; the architecture was revised; models and state engines were developed; backend and frontend integration followed; and validation then became the mechanism for deciding what could legitimately be claimed.

18.4 Lessons from the Engineering Process

- A technically correct component can still be the wrong abstraction for the overall system.
- Digital-twin work requires explicit state and synchronization, not only visualization.
- Model complexity should be justified by evaluation evidence.
- Synthetic data is valuable for integration but cannot substitute for field validation.
- Documentation and evidence classification are part of engineering quality, not administrative overhead.

19. Limitations and Risk Analysis

19.1 Data Limitations

The current project combines heterogeneous and synthetic data, which introduces limitations in representativeness and generalization. Public datasets may not match the exact regional, varietal or sensor characteristics of a target farm. Synthetic telemetry can test system behaviour but cannot establish field-level distributional validity.

19.2 Model Limitations

The grape-yield model currently provides weak held-out performance. The telemetry trigger model performs strongly on its defined test set, but its target is tied to the project’s digital-twin telemetry representation. Broader datasets, external validation, calibration and uncertainty analysis are required before operational deployment.

19.3 Twin Calibration Limitations

State-transition parameters and stage-aware thresholds are engineering representations. They require calibration against real agricultural observations before the twin can be interpreted as a validated physical digital twin. The current validation demonstrates software correctness under defined scenarios, not physical equivalence to a specific vineyard.

19.4 Risk Register

Risk	Impact	Current mitigation	Future mitigation
Synthetic-to-field gap	High	Explicit evidence classification	Real telemetry + field calibration
Weak yield model	High	Not promoted as production-ready	More data, features, temporal validation
Model drift	Medium/High	Evaluation artifacts	Monitoring + scheduled retraining
State parameter error	High	Deterministic tests	Agronomic calibration
Stale telemetry	High	Timestamped state concept	Freshness and sensor-health monitoring
Backend failure	Medium	Service separation	Retries, health checks and observability
Database growth	Medium	SQLite prototype	Managed database / archival strategy

Table 19.1 — Key project risks and mitigation path.

20. Future Scope and Deployment Roadmap

20.1 Immediate Next Engineering Stage

- Connect verified ML artifacts to the live backend inference boundary.
- Complete the canonical state-to-API-to-frontend data path for representative panels.
- Run repeatable end-to-end scenario tests and retain their outputs.
- Expand model evaluation using more representative agricultural observations.
- Calibrate state transitions and stage thresholds using field measurements.

20.2 Medium-Term Extensions

- Real IoT telemetry ingestion and device-health monitoring.
- Regional and varietal crop intelligence.
- Image-based crop and canopy diagnostics as a separate multimodal data stream.
- Uncertainty estimation and confidence-aware recommendations.
- Production database, model registry and observability infrastructure.
- Scenario comparison and intervention optimization.

20.3 Multimodal Extension

Image intelligence should remain a separate modality rather than being flattened into the tabular master dataset. A future multimodal system can associate an image artifact with farm ID, timestamp, crop stage and twin state while preserving the image itself. This allows computer-vision models to enrich the twin without corrupting the semantics of the structured state.

20.4 Deployment Roadmap

Stage	Objective	Exit evidence
Prototype	Subsystems and architecture	Working state, ML, backend and frontend components
Integrated prototype	One complete decision loop	Traceable input → state → model → API → UI output
Pilot	Real farm telemetry	Field calibration and operational monitoring
Validated deployment	Reliable agricultural decision support	External validation, security and operational readiness

Table 20.1 — Deployment maturity model for AgriIntel.

21. Final Evaluation and Conclusion

21.1 Objective-wise Completion

Objective	Status	Evidence / qualification
Study agricultural problem and system context	Achieved	Research, scope refinement and system architecture chapters.
Build heterogeneous data foundation	Achieved / ongoing refinement	Soil, weather, crop, farm, sensor and intervention datasets.
Develop digital-twin state representation	Achieved as prototype	State vector, transition logic and persistence.
Model agricultural interventions	Achieved as simulation	Hydrogel and mulching state behaviour.
Track crop lifecycle	Achieved as prototype	GDD-driven grape phenology state.
Develop predictive ML	Achieved at notebook level	Yield and telemetry models with retained metrics.
Build backend integration	Achieved as prototype	FastAPI service boundary and SQLite persistence.
Build frontend	Achieved as prototype	Multi-panel dashboard evidence.
Validate end-to-end field performance	Not achieved	Requires real telemetry and external field validation.

Table 21.1 — Final objective-wise completion assessment.

21.2 Principal Technical Contributions

- A state-centric interpretation of the agricultural digital twin.
- A deterministic simulation mechanism for evolving soil moisture and interventions.
- Stage-aware grape lifecycle representation using GDD.
- A complementary architecture in which state models and ML have distinct responsibilities.
- A lightweight backend and persistence layer connecting computational services to the frontend.
- A real multi-panel dashboard prototype representing the intended farmer-facing product surface.
- A validation framework that distinguishes software correctness from physical field validation.

21.3 Conclusion

Apollo AgriVerse — AgriIntel has progressed from an initial agricultural intelligence concept into a structured, technically integrated prototype. The project now contains a data foundation, explicit digital-twin state representation, state-transition simulations, lifecycle intelligence, predictive ML pipelines, backend service boundaries, SQLite persistence and a functional dashboard surface. The most important architectural result is the separation of temporal state from predictive intelligence, allowing each mechanism to be evaluated according to what it is actually designed to do.

The final engineering position is deliberately evidence-driven. The digital-twin validation suite demonstrates deterministic software behaviour under defined scenarios. The telemetry ML model demonstrates strong notebook-level predictive performance, while the yield model exposes the need for further modelling work. The frontend demonstrates product direction and interface implementation, while production field deployment remains a future validation stage. This boundary is not a weakness of the project; it defines a credible path from prototype to validated agricultural intelligence.

Final position: AgriIntel is a digital-twin-ready agricultural intelligence prototype with an established architecture and demonstrated subsystem evidence. The next stage is not to add features indiscriminately, but to close the evidence loop through verified model integration, real telemetry, calibration, field validation and deployment hardening.

22. References

- [1] Apollo Continuum, “Apollo AgriVerse — Mission II: AgriIntel Work Package,” official project workspace.
- [2] Apollo AgriVerse project working documentation, “Digital Twin Architecture & ML Manual,” internal technical document.
- [3] Apollo AgriVerse project working documentation, “Complete Architecture & Technical Guide: Database Storage, Telemetry Engine, Digital Twin Physics & Validation.”
- [4] Internal AgriIntel datasets, notebooks, simulation scripts, backend services and frontend artifacts developed during the internship.
- [5] Scikit-learn documentation, model implementations and evaluation utilities used by the project.
- [6] FastAPI documentation, service framework used for the prototype REST boundary.
- [7] SQLite documentation, embedded relational database used for prototype persistence.
- [8] Public agricultural datasets used by the project; dataset-specific creators and repositories should be retained in the final submission bibliography.

Bibliographic note. Before final institutional submission, replace generic internal/public-dataset descriptions with the exact dataset authors, titles, repository URLs/DOIs and access dates used by the team.

A. Technical Evidence Classification

The final report uses the following evidence taxonomy so that an evaluator can distinguish a working implementation from a validated agricultural claim.

Claim type	Permitted evidence	Example in AgriIntel
Implemented	Executable code / retained artifact / working subsystem	State simulator and dashboard prototype
Evaluated	Repeatable test or documented model evaluation	ML test metrics; five twin tests
Simulated	Synthetic or rule-generated evidence	Hydrogel intervention trajectory
Prototype	UI/service demonstration without field validation	Final dashboard panels
Proposed	Architecture or future roadmap	Real IoT deployment

Table 0.2 — Evidence taxonomy used for final project claims.

A.1 Evidence Boundary

The presence of a number in the report does not automatically make the underlying phenomenon field-validated. Model metrics belong to a specific dataset and split; simulation values belong to defined state equations; frontend values belong to the computational source connected to the panel. The report intentionally preserves these distinctions.

B. Representative State and Data Schema

Entity	Representative fields	Relationship / purpose
Farm	farm_id, region, configuration	Parent identity for all contextual records.
Soil	pH, N, P, K, moisture	Substrate state and agronomic context.
Weather	date, Tmax, Tmin, rainfall, humidity	External forcing and GDD calculation.
Telemetry	timestamp, air/soil temperature, humidity, moisture	Dynamic state synchronization.
Hydrogel	capacity, release	Intervention state and effect.
Mulch	degradation, cooling effect	Surface intervention state.
Crop	crop_id, variety, stage, cumulative GDD	Lifecycle context.
Intelligence	prediction, risk, recommendation	Derived decision context.

Table 0.3 — Representative entities and fields used by the AgriIntel state/data architecture.

B.1 Temporal Semantics

Time is a first-class dimension in the twin. State transitions occur in sequence; telemetry and weather inputs are associated with a timestep; lifecycle progression depends on accumulated environmental exposure; and persisted state provides recovery and replay context.

C. Digital Twin Validation Evidence

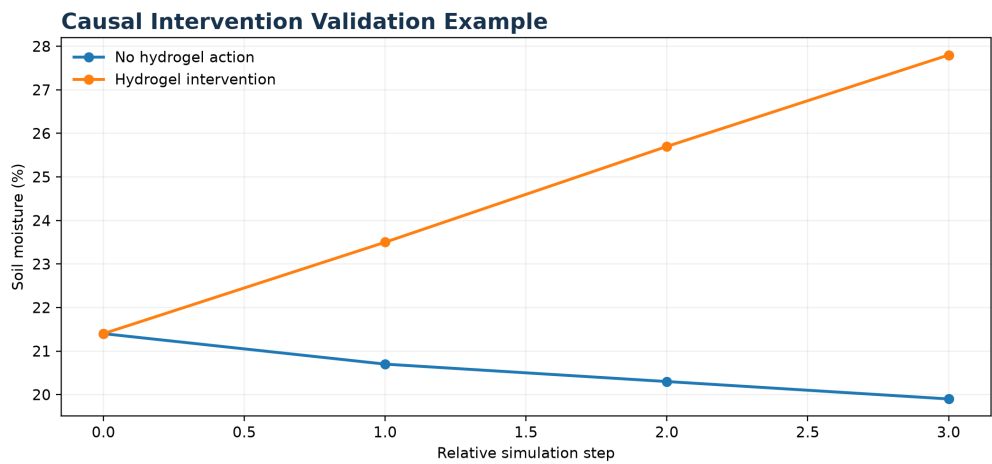


Figure C.1 — Causal intervention evidence rendered from the documented validation values.

State Model Abstraction

State models provide explicit temporal memory; ML models provide learned relationships where appropriate.

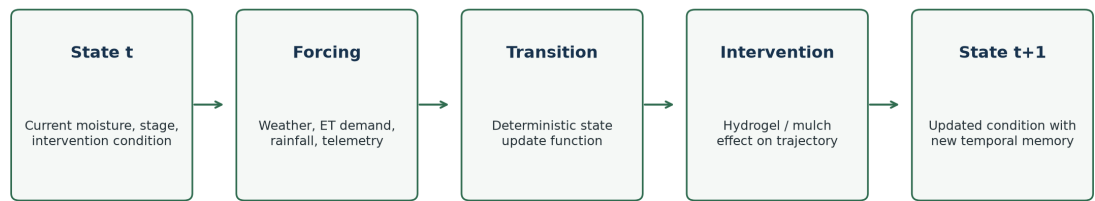


Figure C.2 — State-transition abstraction used in the validation architecture.

The validation suite should be retained alongside the source code and its generated plots. A production evidence package should additionally retain the exact input seed/configuration, code version, dataset identity and environment version for every accepted run.

D. Frontend Evidence Register

Asset	Detected path	Included in report?
Alert Panel	assets\end_term>alert_panel.jpeg	Yes
Analytics Panel	assets\end_term\analytics_panel.jpeg	Yes
Digital Twin Panel	assets\end_term\digital_twin_panel.jpeg	Yes
Hydrogel Panel	assets\end_term\hydrogel_panel.jpeg	Yes
Intelligent Soil Panel	assets\end_term\intelligent_soil_panel.jpeg	Yes
Lifecycle Panel	assets\end_term\lifecycle_panel.jpeg	Yes
Prediction Panel	assets\end_term\prediction_panel.jpeg	Yes
Report Panel	assets\end_term\report_panel.jpeg	Yes
Setting Panel	assets\end_term\setting_panel.jpeg	No — retained as project evidence
Simulation Panel	assets\end_term\simulation_panel.jpeg	No — retained as project evidence
Smart Mulching Panel	assets\end_term\smart_mulching_panel.jpeg	No — retained as project evidence
Weather Panel	assets\end_term\weather_panel.jpeg	No — retained as project evidence
Dashboard Panel 1	assets\images\panel1.jpeg	No — retained as project evidence
Dashboard Panel 10	assets\images\panel10.jpeg	No — retained as project evidence
Dashboard Panel 11	assets\images\panel11.jpeg	No — retained as project evidence
Dashboard Panel 12	assets\images\panel12.jpeg	No — retained as project evidence
Dashboard Panel 2	assets\images\panel2.jpeg	No — retained as project evidence
Dashboard Panel 3	assets\images\panel3.jpeg	No — retained as project evidence
Dashboard Panel 4	assets\images\panel4.jpeg	No — retained as project evidence
Dashboard Panel 5	assets\images\panel5.jpeg	No — retained as project evidence
Dashboard Panel 6	assets\images\panel6.jpeg	No — retained as project evidence
Dashboard Panel 7	assets\images\panel7.jpeg	No — retained as project evidence
Dashboard Panel 8	assets\images\panel8.jpeg	No — retained as project evidence
Dashboard Panel 9	assets\images\panel9.jpeg	No — retained as project evidence

Table 0.4 — Detected final frontend screenshot assets and their use in this report.

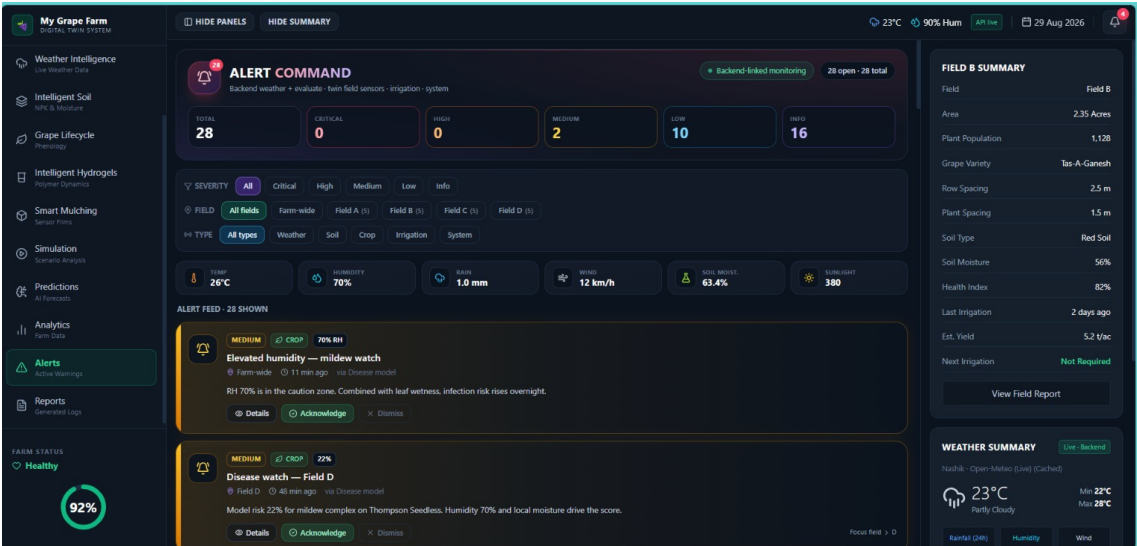


Figure D.1 — Alert Panel — final frontend evidence.

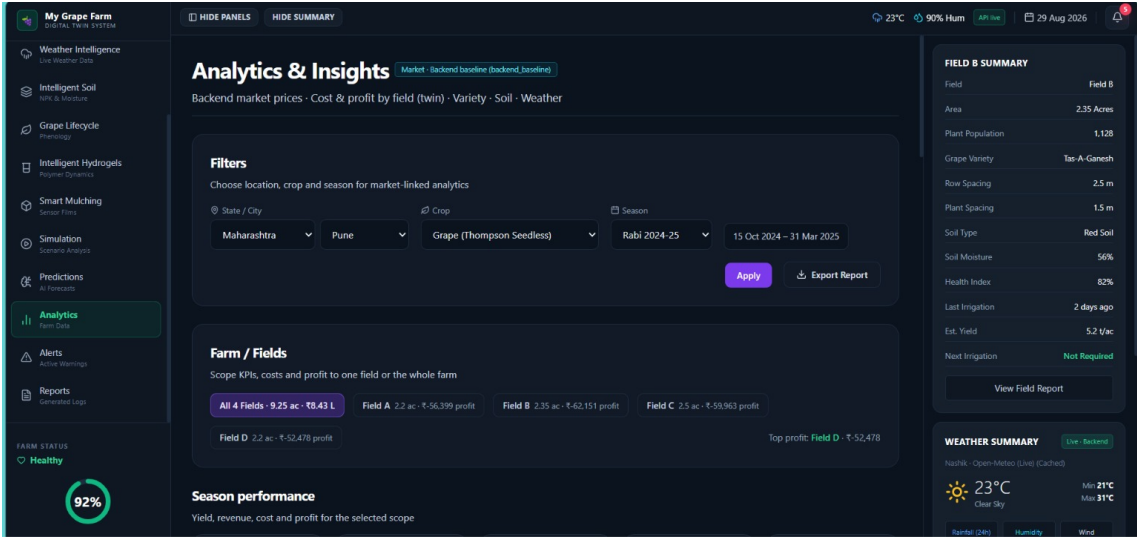


Figure D.2 — Analytics Panel — final frontend evidence.

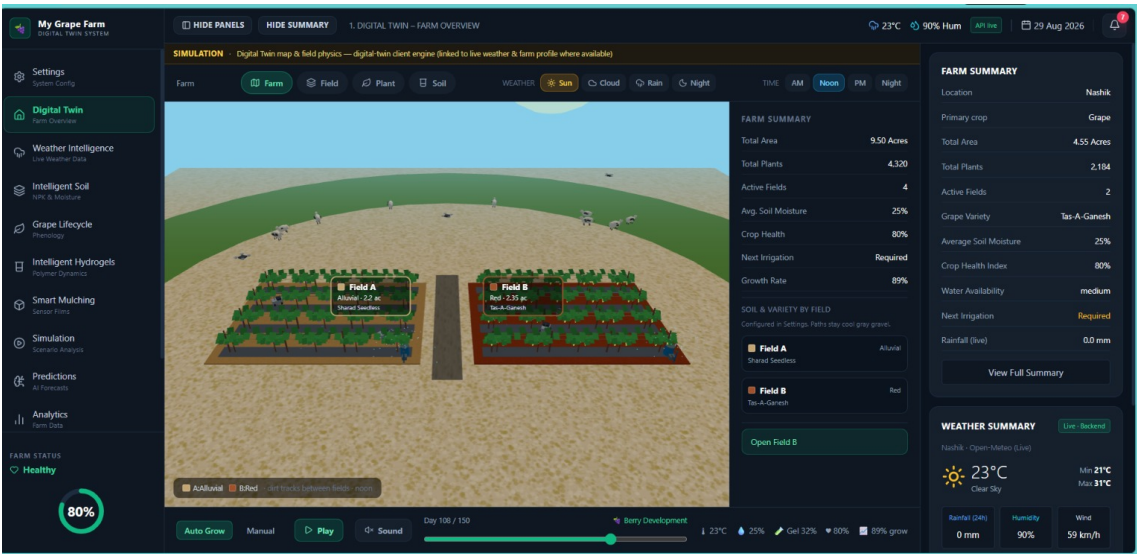


Figure D.3 — Digital Twin Panel — final frontend evidence.

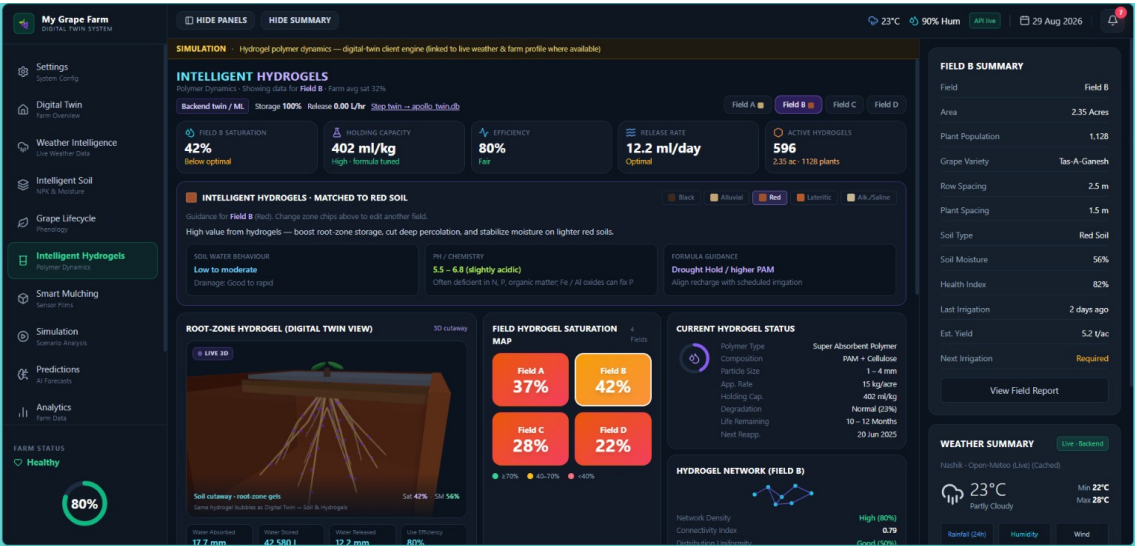


Figure D.4 — Hydrogel Panel — final frontend evidence.

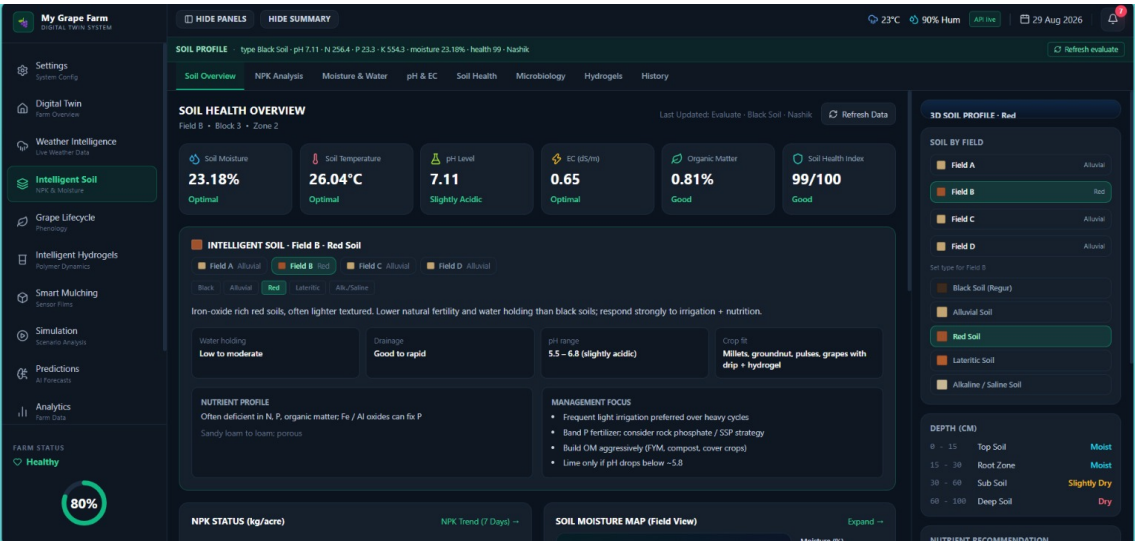


Figure D.5 — Intelligent Soil Panel — final frontend evidence.

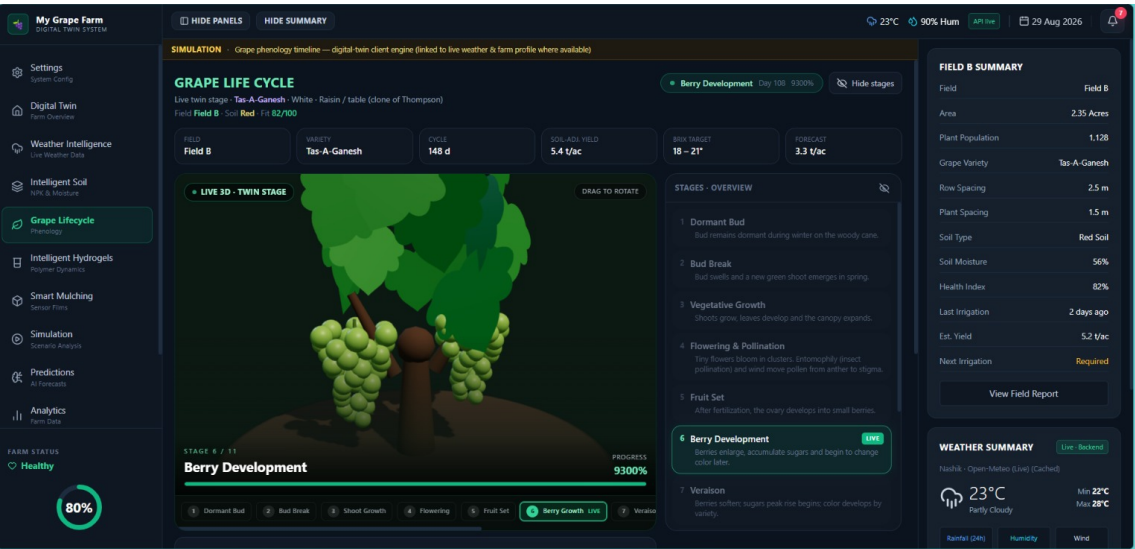


Figure D.6 — Lifecycle Panel — final frontend evidence.

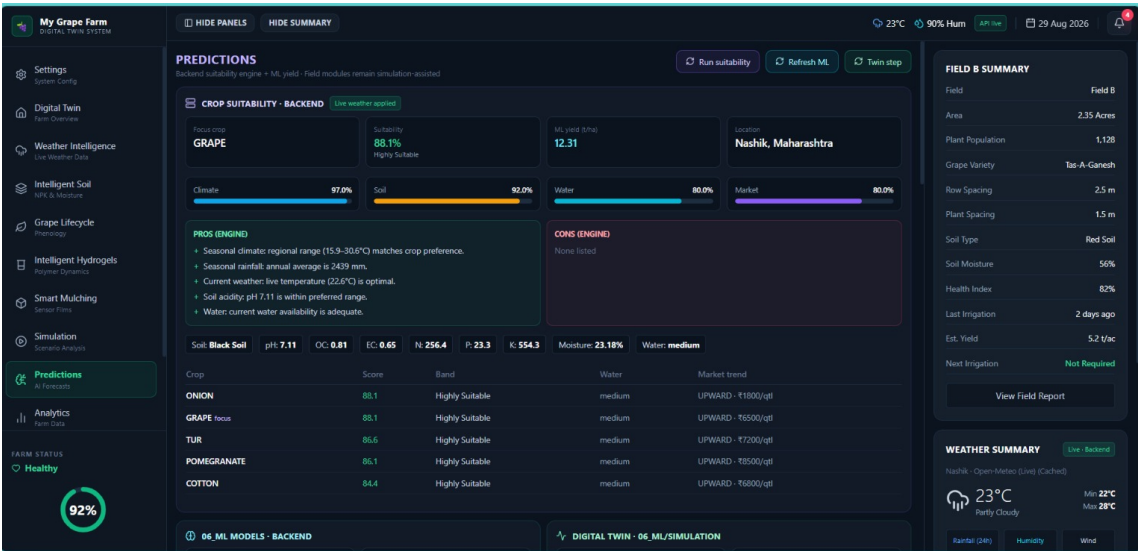


Figure D.7 — Prediction Panel — final frontend evidence.

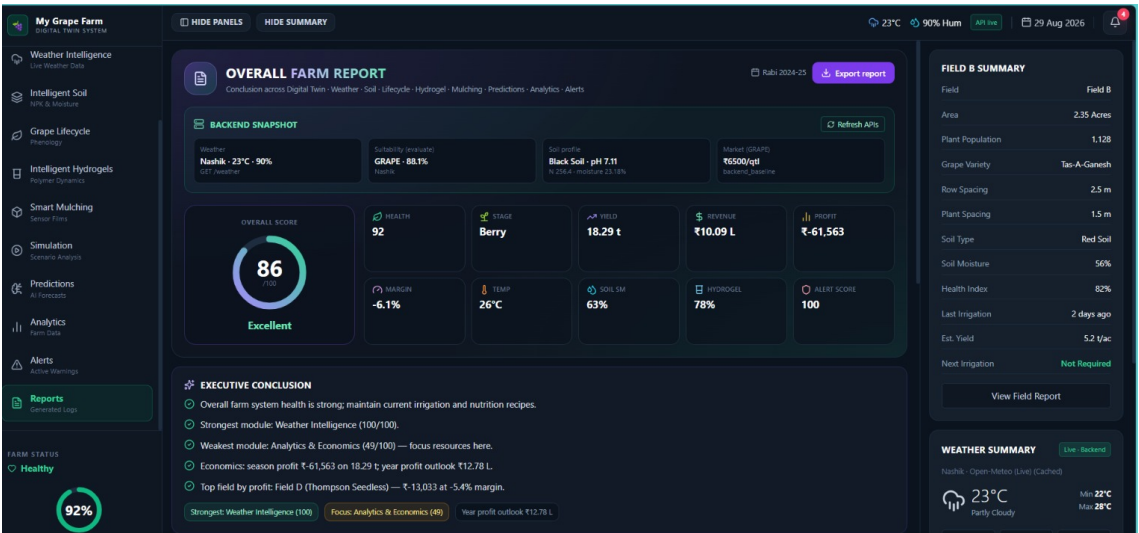


Figure D.8 — Report Panel — final frontend evidence.

E. Model Evaluation Evidence

Model	Target	Data	Metrics	Interpretation
Random Forest	yield_tons_ha	800 rows; 640/160 split	CV R ² -0.0944 ± 0.0299; Test RMSE 0.6134; MAE 0.4569; R ² -0.0942	Weak baseline; not production-ready
Gradient Boosting	hydrogel_water_storage_pct	1,000 rows; 800/200 split	RMSE 0.9484%; MAE 0.1456%; R ² 0.9900	Strong notebook result; broader validation required

Table E.1 — Final documented ML evaluation evidence.

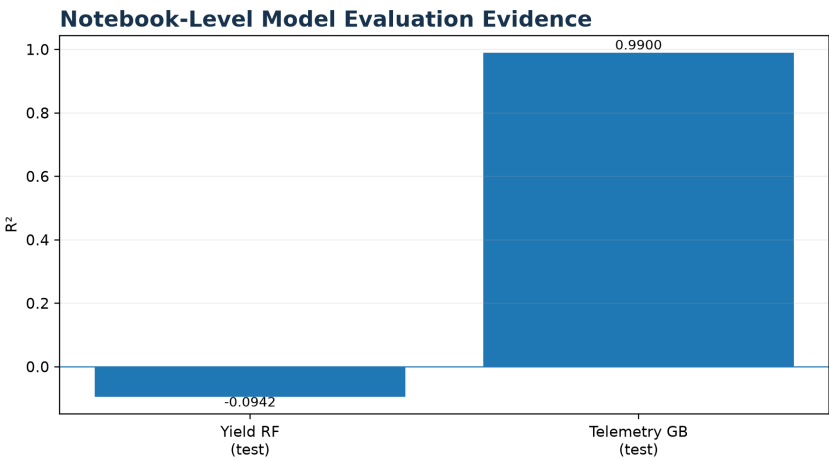


Figure E.1 — Model evaluation summary plot.

F. Backend and Persistence Evidence

Backend Integration Boundary

The backend orchestrates services; it should not duplicate the domain logic owned by the twin or model layer.



Figure F.1 — Backend service boundary.

SQLite Persistence Model

The documented recovery path demonstrates why persistence is part of the digital-twin implementation, not an optional add-on.

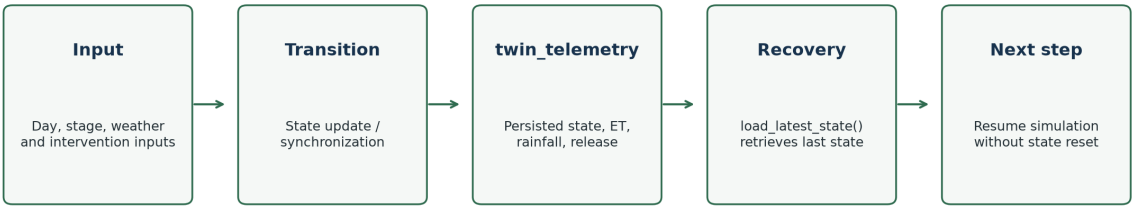


Figure F.2 — SQLite persistence and recovery flow.

```
uvicorn main:app --reload --port 8000
```

```
SELECT * FROM twin_telemetry ORDER BY day DESC LIMIT 1
```

These commands document the local development and state-recovery paths described by the project technical guide. They should be retained with the backend source and test suite.

G. Final Frontend Gallery

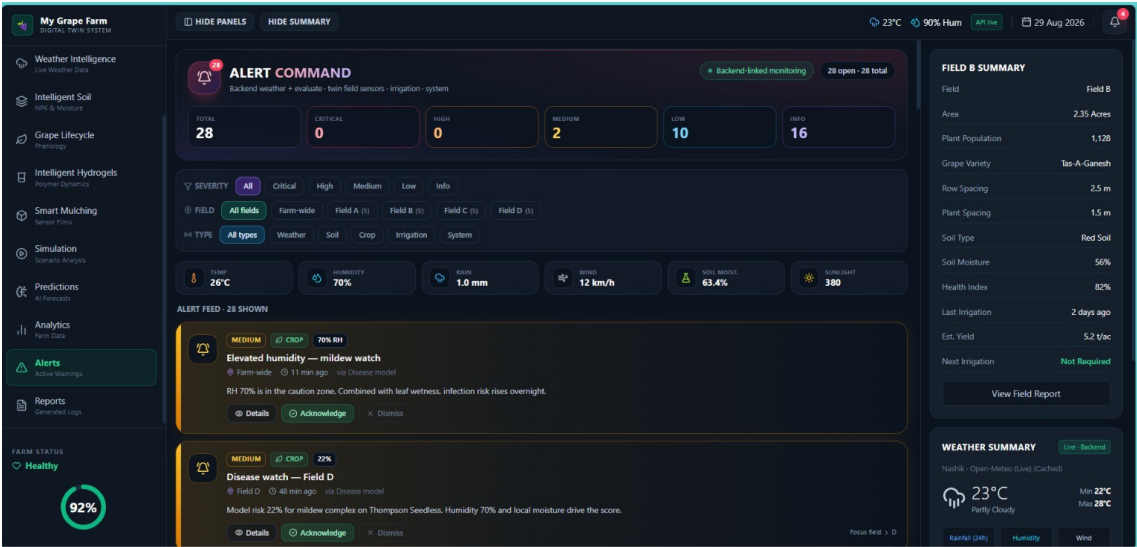


Figure G.1 — Alert Panel — final dashboard view.

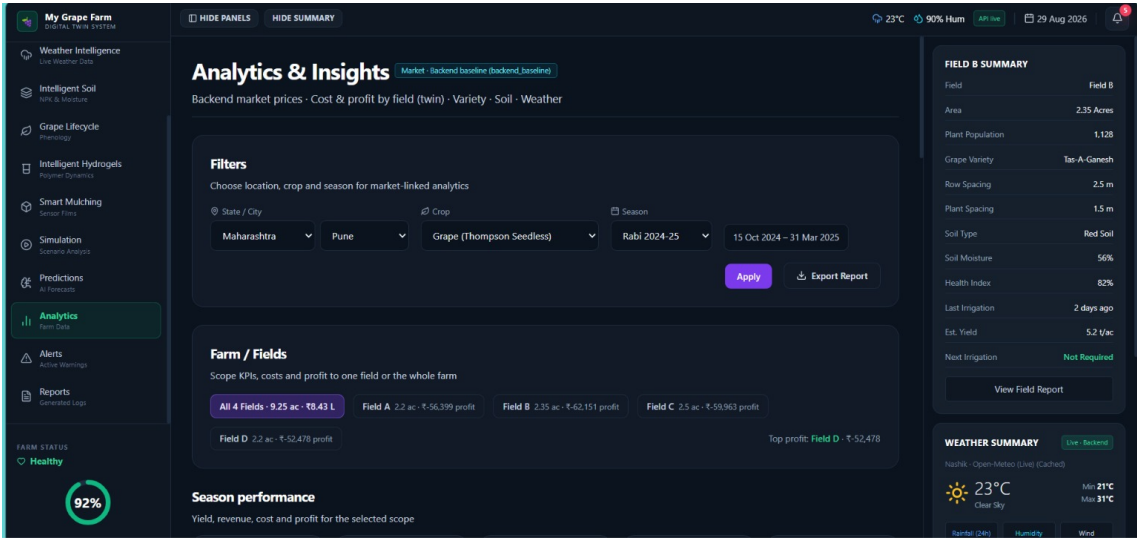


Figure G.2 — Analytics Panel — final dashboard view.

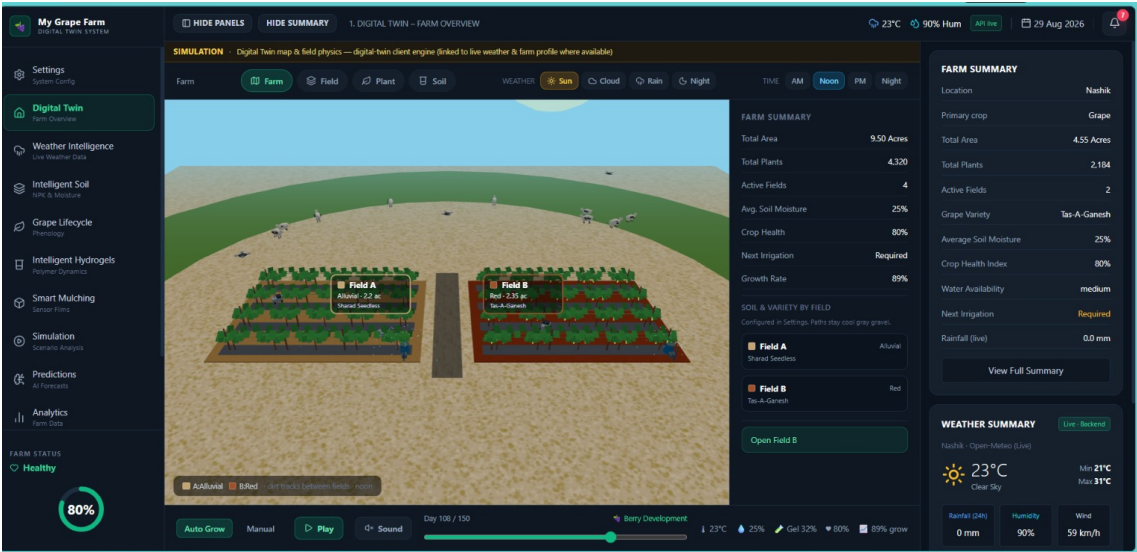


Figure G.3 — Digital Twin Panel — final dashboard view.

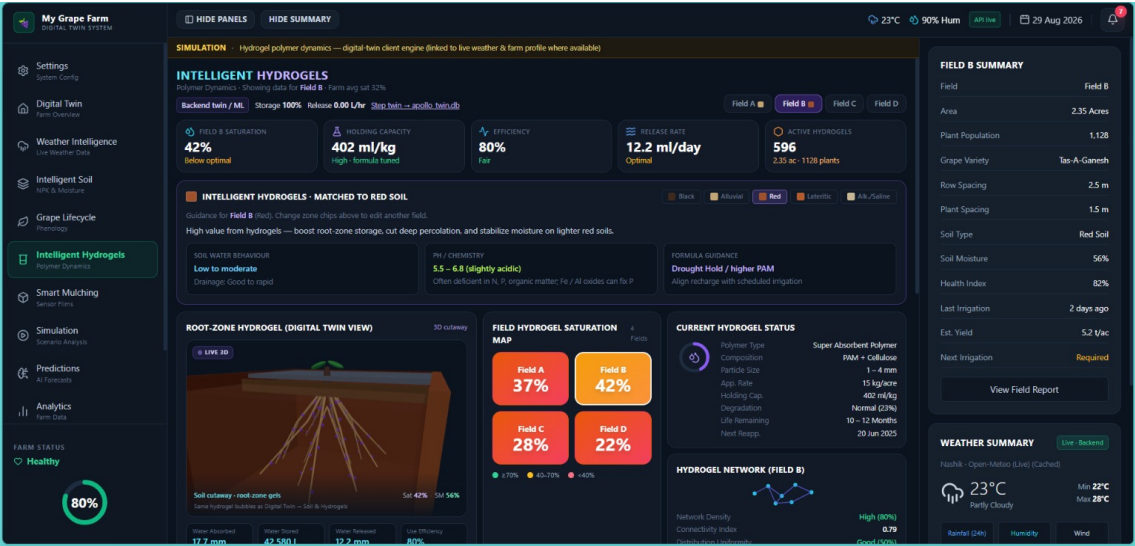


Figure G.4 — Hydrogel Panel — final dashboard view.

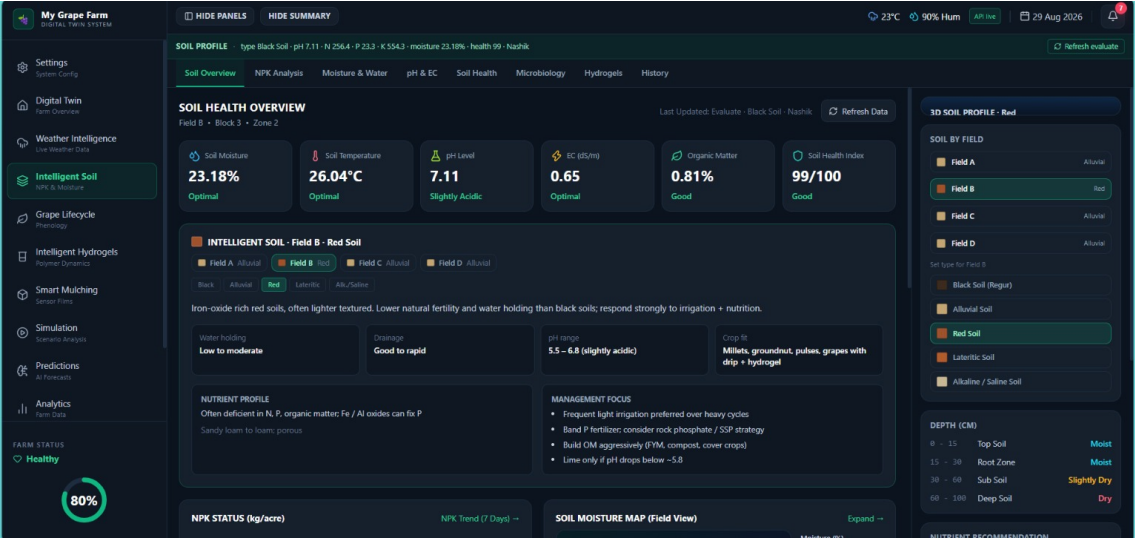


Figure G.5 — Intelligent Soil Panel — final dashboard view.

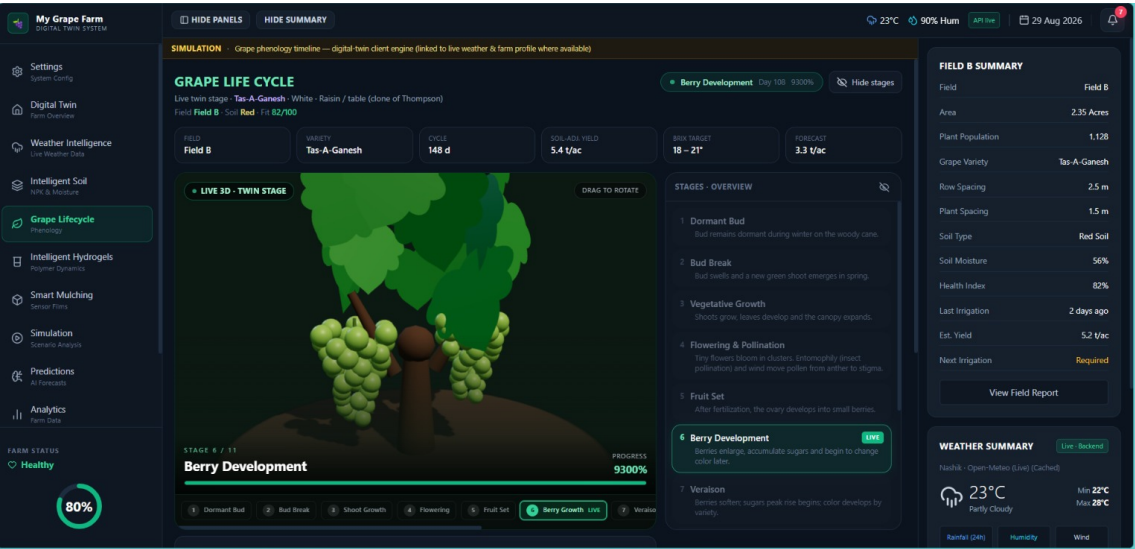


Figure G.6 — Lifecycle Panel — final dashboard view.

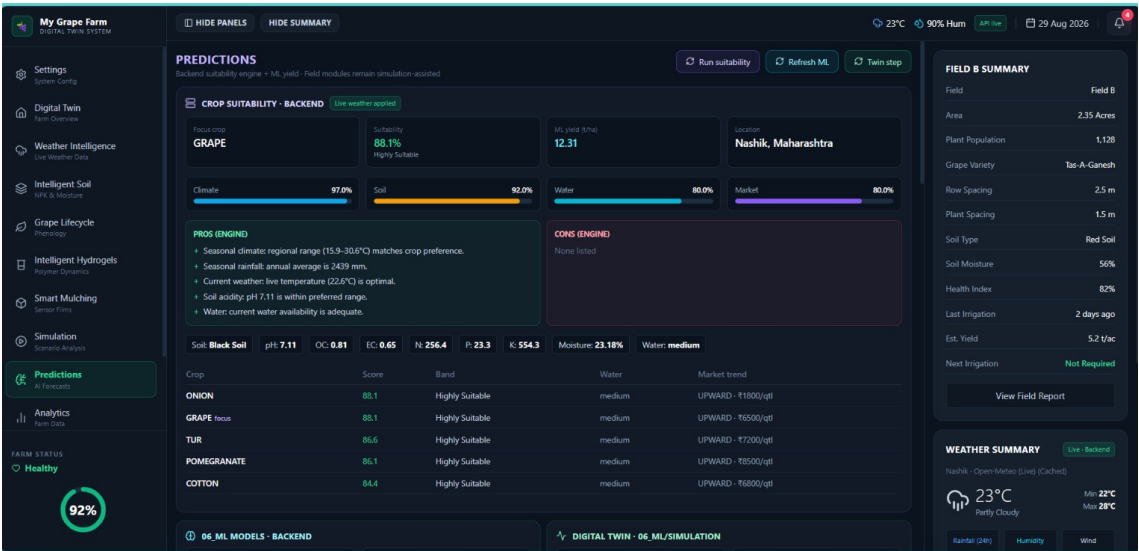


Figure G.7 — Prediction Panel — final dashboard view.

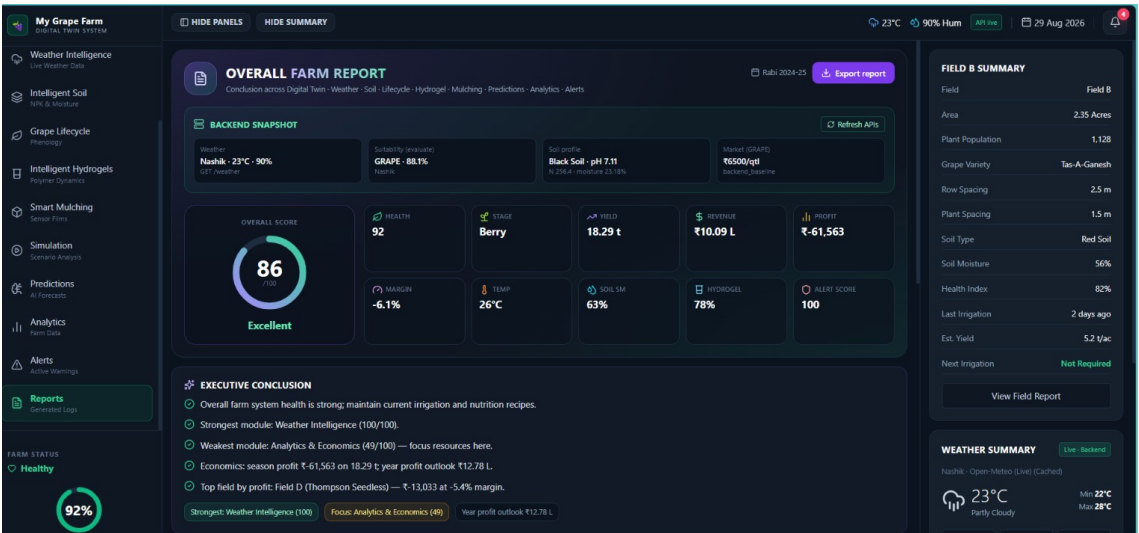


Figure G.8 — Report Panel — final dashboard view.

H. Deployment Readiness Checklist

Readiness area	Current state	Required next evidence
Data	Prototype datasets prepared	Provenance, field representativeness and governed ingestion
Twin	State engine + validation suite	Agronomic calibration and real telemetry replay
ML	Two evaluated notebook pipelines	External validation, model registry and drift monitoring
Backend	FastAPI + SQLite prototype	Authentication, observability, production database and deployment
Frontend	Functional dashboard prototype	Live canonical API wiring and UI integration tests
Operations	Local/reviewable execution	Deployment automation, monitoring and failure recovery

Table H.1 — Deployment-readiness checklist for transition beyond the internship prototype.

The final end-term deliverable should therefore be evaluated as a complete engineering package with clear evidence boundaries. The roadmap is intentionally framed as an evidence closure plan rather than a feature wishlist.